Undergraduate Project Report
2025/26

From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol

Name:	Xiheng LI
School:	International School
Class:	2022215107
QMUL Student No.:	221166471
BUPT Student No.:	2022213192
Programme:	Telecommunications Engineering with Management

Date: 27-04-2026

Table of Contents

Abstract 3

Keywords 3

Chapter 1: Introduction 5

1.1 Problem Statement 5

1.2 Our Solution 6

1.3 Main Contributions 7

Chapter 2: Background 8

2.1 Agent Systems: Prior Work 8

2.2 What MCP Provides 8

2.3 Limits of User-space Mediation 9

2.4 Relation to Native Linux Security Mechanisms 9

2.5 Related Work 10

2.6 Threat Model 11

Chapter 3: Design and Implementation 14

3.1 Design Goals 14

3.2 System Architecture 14

3.3 Control-Plane Workflow 16

3.4 Wire Protocols 18

3.5 Manifest Semantics and Tool Identity 20

3.6 Session Management and Approval Binding 21

3.7 Kernel Module Design 23

3.8 Daemon Implementation 25

3.9 State Reconciliation and Recovery 26

3.10 Rationale for the Kernel--User-Space Split 27

3.11 Simplifications and Optimisation Opportunities 28

Chapter 4: Results and Discussion 29

4.1 Latency and Control-Plane Overhead 29

4.2 Throughput and Scalability 34

4.3 Registry Scaling 35

4.4 Security Evaluation 36

4.5 Extended Threat Model 38

4.6 Daemon Failure and Recovery 40

4.7 Discussion 41

4.8 Threats to Validity **42**

Chapter 5: Conclusion and Further Work **43**

5.1 Conclusion **43**

5.2 Reflection **44**

5.3 Further Work **44**

References **46**

Acknowledgment **50**

Appendices **51**

Disclaimer **51**

Acknowledgement of using Generative AI **52**

Project specification **53**

Early-term progress report **60**

Mid-term progress report **67**

Supervision log **74**

Additional Appendices **77**

A.1 Wire Protocol Reference 77

A.2 System Comparison Table 80

A.3 Experiment Kernel Interface 80

Risk and Environmental Impact Assessment **84**

Abstract

AI agents increasingly rely on external tools, yet existing tool-invocation frameworks provide little operating-system support for enforcing control-plane safety. When mediation remains entirely in a user-space gateway, key invariants depend on the correctness and availability of that process, including semantic consistency between approved and executed operations, identity binding that resists session confusion and replay, and preservation of approval state across failures.

We present `linux-mcp`, a kernel-assisted control plane for LLM tool invocation on Linux. Rather than moving full tool logic into the kernel, `linux-mcp` keeps semantic parsing and tool execution in user space while moving a narrow set of control-plane responsibilities into a kernel module: admission decisions, approval-ticket lifecycle, and binding verification. This design requires no modification to existing tools beyond a declarative JSON manifest, and makes approval state durable across daemon failure and protected from unprivileged user-space interference.

We evaluate `linux-mcp` against a pure user-space baseline and a hardened seccomp variant across latency, arbitration cost, throughput, registry scaling, attack resistance, and failure recovery. The kernel path enforces all evaluated control-plane invariants, while the user-space and seccomp baselines exhibit bypass rates of 90.9% and 36.4%; an extended-attack rerun surfaced a missing peer-credential check in the stock module, closed by a six-line patch at no measurable latency cost. Performance overhead is small: latency is within measurement noise at small and medium payloads and about 3.4% lower at 1 MB, and a paired microbenchmark attributes only 4.334 μs to the full kernel arbitration path above the measured Netlink noise floor. The prototype supports 43 tools across 14 applications and is publicly available at `https://github.com/xiheng724/linux_mcp`.

Keywords

AI Agents, Operating Systems, Tool Invocation, System Security

摘要

随着 AI agents 日益依赖外部工具来扩展其能力，现有系统在工具调用管理方面仍缺乏高效且安全的操作系统级支持。对大语言模型（LLM）和外部工具的非受控访问会带来资源滥用、安全漏洞以及执行不可靠等风险，这凸显了对 AI 原生操作系统支持的迫切需求。

为此，我们提出了 `linux-mcp`，一种面向 Linux 操作系统的、内核辅助的工具调用控制平面。该系统引入了一种新的操作系统架构：将语义解析和工具执行保留在用户态，同时将准入控制下沉至 Linux 内核模块中。该设计通过统一的守护进程对所有外部工具调用进行中介，将批准状态持久化于内核中，从而在守护进程崩溃后仍可保持控制平面状态，并与非特权进程严格隔离。基于 MCP 协议的抽象框架支持无缝的工具集成：开发者只需在 manifest 中以声明方式描述工具的功能和参数，无需编写任何面向内核的代码。

我们从延迟、仲裁成本、吞吐量、注册表扩展性、抗攻击能力和故障恢复等方面，将 `linux-mcp` 与纯用户态和基于 seccomp 的强化用户态基线进行了对比。实验结果表明，内核路径能阻断全部已测试的控制平面攻击，而两种用户态基线均存在明显绕过率；扩展攻击实验还发现并以六行补丁修复了原始模块中缺失的 peer-credential 校验，无可测延迟开销。性能开销较小：小、中负载延迟与用户态无显著差异，1 MB 下反而约低 3.4%；经配对微基准测试，完整内核仲裁路径仅比 Generic Netlink 底噪多 4.334 μs。该原型在 14 个应用中暴露 43 个工具，已开源：`https://github.com/xiheng724/linux_mcp`。

关键词

AI 智能体，操作系统，工具调用，系统安全

Chapter 1: Introduction

Recent advances in Large Language Models (LLMs) have enabled instruction following, reasoning, and interaction with external environments, forming the foundation for autonomous agentic systems [1–3]. LLM-based agents are now deployed across a wide range of applications, from direct-acting assistants to sophisticated multi-step reasoning and problem-solving systems [4–7]. Model Context Protocol (MCP) has emerged as a common interface between LLM agents and external tools [8].

1.1 Problem Statement

Tool-using, stateful AI agents expose a fundamental mismatch between modern agent frameworks and the abstractions provided by conventional operating systems [6, 9]. While conventional operating systems effectively manage processes, files, and network resources, MCP-style workflows are organized around higher-level concepts such as semantic tool descriptions, session context, approval boundaries, and cross-step state. This mismatch leads to fragile control planes when enforcement remains entirely in user-space. Frameworks such as OpenClaw [10] and Hermes Agent [11] demonstrate that granting agents execution capability and persistent context can lead to security risks including prompt injection, privilege escalation, and cross-step exploitation. Among these, prompt injection originates at the model–input boundary and is orthogonal to the control plane; this work focuses on the latter two classes, where the integrity of approval, identity binding, and cross-step state determines whether a compromised or manipulated request can actually take effect.

A single gateway process is typically responsible for interpreting tool manifests, binding requests to identities, and authorizing execution, effectively placing the control plane in user space. Such a design introduces a single point of failure and expands the trusted computing base, making it vulnerable to crashes, logic bugs, or compromised control paths [12]. Under such a design, three properties become difficult to guarantee reliably: (1) semantic consistency between declared tool behaviour and executed operations, (2) durability of approval state across gateway crashes, and (3) robust identity binding in the presence of replay and session confusion [5, 13].

This leads to a concrete question: which control-plane invariants in MCP-style tool invocation benefit from kernel-level enforcement, and can that enforcement improve security and resilience without making the system unacceptably slow?

1.2 Our Solution

`linux-mcp` addresses this by moving admission control, approval state, binding checks, and durable observability (i.e., kernel-resident audit records that survive gateway restart) into a Linux kernel module, while leaving semantic parsing and tool execution in user space. This is not a full AI-native operating system; the kernel role is deliberately narrow, enforcing only the invariants user-space cannot reliably hold.

The architecture has four layers: an LLM client (`llm-app`), a manifest-aware gateway (`mcpd`), a Generic-Netlink-based kernel control plane (`kernel_mcp`), and resident tool services over Unix domain sockets. Every tool invocation follows a fixed path through all four layers; no request reaches a tool service without first obtaining an explicit kernel admission decision. The component responsibilities are detailed in Section 3.2, and the rationale for the kernel–user-space split is developed in Section 3.10.

This split partitions the gateway's trust along a time axis. At initialisation, `mcpd` is trusted to populate the kernel registries with manifest-derived tool identities and to bind sessions to peer credentials. At run time, it is no longer fully trusted: every `TOOL_REQUEST` is independently re-checked in the kernel, so a session-id mismatch, a skipped ticket transition, or a TOCTOU race in the daemon is rejected rather than admitted. Registry entries, approval tickets, and binding checks therefore reside in the kernel, gated by `CAP_NET_ADMIN`-protected Generic Netlink access, durable across daemon failure, and inaccessible to unprivileged processes.

This position is deliberately narrower than either extreme in prior work. AIOS-style proposals that reimagine OS abstractions for agents, at the cost of deployability on existing systems [6]. User-space frameworks preserve deployment flexibility but inherit the structural weaknesses identified in Chapter 2. `linux-mcp` instead moves only those invariants that require protection *from the gateway itself* into the kernel, leaving semantic parsing, tool execution, and everything else in user space. Nor does it replace existing Linux security mechanisms such as seccomp or LSM; it complements them at a layer those mechanisms cannot reach. The central hypothesis is that this targeted placement improves binding safety and failure resilience at acceptable performance cost; Chapter 4 tests this claim empirically.

Comparison with user-space systems. To contextualise the design, we compare `linux-mcp` with two representative user-space tool-invocation frameworks: OpenClaw [10], which uses a monolithic user-space gateway, and Hermes Agent [11], which runs the entire control pipeline inside a single Python process, with tool execution dispatched to in-process handlers or MCP subprocess servers. Table 1 highlights the key architectural differences.

Feature	linux-mcp	OpenClaw	Hermes Agent
Control-plane placement	Kernel-assisted	User-space gateway	In-process (same PID as agent loop)
Trust boundary	Split (user + kernel)	Monolithic (user-space)	Python process; optional Docker sandbox
Approval enforcement	Ticket-based (kernel)	User-space logic	Per-session in-memory module state
Binding correctness	Kernel-verified (UDS peer creds)	Gateway-managed	Session-id only; no OS-level binding
Failure resilience	State persists in kernel	Lost on gateway failure	Approval state lost on restart

Table 1: Key design differences between `linux-mcp`, OpenClaw, and Hermes Agent.

1.3 Main Contributions

This paper makes the following contributions:

- **A kernel-resident control-plane for tool invocation.** We implement `kernel_mcp`, a Linux kernel module that maintains tool and agent registries, arbitrates invocation decisions, and manages the approval-ticket lifecycle, all behind a `CAP_NET_ADMIN`-gated Generic Netlink interface. The module holds no JSON parser and executes no tool code; its sole role is to keep admission state durable across daemon failure and unreachable from unprivileged processes.

- **A manifest-aware gateway that bridges semantics and enforcement.** We build `mcpd`, a user-space daemon that derives semantic hash identifiers from tool manifests, reconciles them with the kernel registry, anchors sessions to UDS peer credentials, and validates every request against its declared JSON schema before routing it through the kernel mediation path. `mcpd` concentrates all manifest-handling complexity in one place, leaving both the kernel and the LLM client unaware of it.

- **An empirical evaluation of kernel placement as a design choice.** We compare `linux-mcp` against pure and hardened (seccomp) user-space baselines across latency, throughput, attack resistance, and daemon failure. The results quantify the cost of moving admission into the kernel, demonstrate that the specified control-plane invariants hold under a targeted attack set (30 repetitions of 11 attack cases across four categories), and show that kernel-held approval state survives daemon failure in a way neither user-space baseline can match.

Chapter 2: Background

2.1 Agent Systems: Prior Work

Modern operating systems are built around the process as the primary unit of protection: address-space isolation, controlled file access, and well-defined IPC [14]. These abstractions remain fundamental, but agent-based workloads introduce a layer of intent between the user and execution that they do not directly address. An agent plans over semantically described tools, constructs structured requests, and acts on behalf of a user. Correctness therefore depends not only on process isolation, but also on whether the requested semantics match the operation that is actually executed.

Several lines of work have responded to this gap. Systems such as generative agents demonstrate long-lived entities capable of maintaining memory and coordinating workflows [4]. Frameworks including LangChain [15], LangGraph [16], AutoGPT [17], CrewAI [18], and Semantic Kernel [19] structure tool composition in multi-step pipelines. More ambitious proposals embed LLM-centric services directly into a system-level kernel managing scheduling, memory, and tool access [6, 7, 9, 20]. At the application layer, OpenClaw [10] acts as a persistent user-space control layer for tool execution, while OpenHarness [21] and Hermes Agent [11] integrate control logic directly into the agent process itself; Harness [22] embeds governed AI agents in CI/CD pipelines; Apple Intelligence [23] integrates agents with on-device OS services and a private-cloud compute layer.

Section 2.3 details why this enforcement placement matters.

2.2 What MCP Provides

The Model Context Protocol (MCP), released by Anthropic in November 2024, provides a useful abstraction for structuring tool-mediated agent interaction [8]. It defines a client-host-server architecture in which tools, resources, and prompts are exposed through a standardised JSON-RPC interface. The host coordinates sessions and mediates access, while servers implement capabilities; tools are the primary mechanism by which agents take executable actions on external systems. MCP rapidly achieved broad ecosystem adoption, with implementations in Python, TypeScript, C#, and Java, and integration across major LLM providers and agent frameworks [24].

This separation simplifies interoperability and modularity. However, it also concentrates responsibility in the host: identity management, approval decisions, and consistency between tool descriptions and execution all depend on correct host behavior. MCP defines how components communicate, but does not specify how a local system should enforce these properties. `linux-mcp` implements the enforcement layer that MCP leaves unspecified: it provides the local

mechanism that turns the protocol's abstract trust model into a set of kernel-verified invariants.

2.3 Limits of User-space Mediation

Semantic drift. Tool invocation depends on semantically rich manifests combining natural-language descriptions, schemas, and examples [3, 8, 25]. Because both interpretation and enforcement reside in a mutable user-space process, a change in implementation or a weakened control path may result in executing a capability that no longer matches the approved semantics [26].

Identity anchoring. Agent systems bind requests to sessions or agent identifiers, not PIDs [1, 4, 17], and MCP provides no mechanism to anchor these identities in OS-visible credentials. Prompt injection and session confusion attacks can therefore cause agents to exceed their intended authority even when the gateway appears to enforce correct behaviour [13, 27, 28].

Durability. Approval decisions and pending requests may be lost on crash or restart when control-plane state is held only in memory [9]. The fragility is particularly acute in long-running agent systems where execution spans multiple steps and decisions [29, 30].

These limitations are not unique to any single framework; they reflect a broader pattern in which control-plane invariants [31] are difficult to enforce when confined to user space [32–35].

2.4 Relation to Native Linux Security Mechanisms

Any such design must coexist with existing Linux security mechanisms. Facilities such as seccomp, AppArmor, SELinux, and Landlock provide strong guarantees over system calls, resources, and process privileges [36–40]. These mechanisms operate at the level of kernel objects and access rights.

However, none of them capture higher-level concepts such as tool identity, semantic equivalence between a manifest and an execution endpoint, or the scope of an approval decision. The root cause is a layer mismatch: existing mechanisms enforce policy at the level of kernel objects (files, processes, syscalls), whereas tool identity is an application-layer concept defined by a semantic hash of a JSON manifest. Extending these mechanisms to enforce tool-identity would require either encoding tool hashes into LSM policy or hooking into the application-protocol layer, which breaks the principle that LSM hooks attach to kernel operations, not to application messages.

This gap motivates introducing a dedicated control layer that validates semantic and identity invariants at the point of tool execution, outside the mutable execution path of the user-space gateway.

2.5 Related Work

Several existing mechanisms address related but distinct problems; understanding why they fall short for MCP-style enforcement motivates the present design.

Existing kernel-native mechanisms such as seccomp-BPF, eBPF/LSM hooks, Landlock, and capability-based designs provide strong control over syscalls, kernel objects, and process-scoped authority [33, 36, 40–44]. However, they operate below the application-protocol layer and therefore do not natively express MCP-specific concepts such as manifest-derived tool identity or semantic equivalence between a declared tool and an executed endpoint. A tool's identity in this system is a SHA-256 digest of its JSON manifest, a concept that does not exist at the syscall or VFS layer; no hook in seccomp or LSM fires at the point where a JSON-RPC payload is interpreted. Modern eBPF can perform bounded map lookups and keep per-session state in pinned maps, so neither hash checks nor ticket storage is the hard part. The harder part lies at the attachment point. eBPF LSM hooks fire on kernel operations such as socket receive and file open, not on parsed JSON-RPC messages, so any in-kernel enforcer would have to re-parse MCP framing and reconstruct tool identity from raw socket bytes on every hook invocation, all under the eBPF verifier's bounded-loop and instruction-count constraints. For this reason, these mechanisms can harden the surrounding execution environment, but they do not directly enforce the control-plane invariants targeted in this work.

The agent frameworks surveyed in Section 2.1 locate the full control plane in user space; correctness depends entirely on the in-process runtime remaining uncompromised. OpenClaw [10] operationalises this pattern at scale, and independent security analyses confirm the resulting structural vulnerabilities [27, 28]. Industrial deployments such as Harness [22] add platform-level Role-Based Access Control (RBAC) on top, but the trust anchor is relocated to a cloud control plane rather than eliminated. Recent systems on application-layer mandatory access control including privilege-escalation defences for LLM agents [34] and programmable privilege control for tool-calling agents [35] have enforced policy inside a modified agent runtime, so invariant correctness depends on the runtime itself remaining intact. Most of these approaches keep the trust anchor in user-space or in a cloud-hosted control plane; their correctness degrades when the runtime or gateway is compromised. Apple Intelligence is the exception, it anchors trust in the Secure Enclave and PCC attestation, but that hardware-rooted model is not available to general Linux MCP deployments [23].

AIOS-style proposals such as AIOS, OS-Copilot, MemGPT, and AgentOS [6, 7, 9, 20] pursue a broader scope: they embed LLM-specific abstractions—scheduling, memory management, tool access—into a new OS substrate. These proposals broaden scope by redesigning OS infrastructure; `linux-mcp` instead adds a narrow kernel role to an otherwise unchanged Linux system, targeting a bounded set of MCP-style control-plane invariants.

Attack class	Representative attack	Defence in `linux-mcp`
Spoofing	Forged `session_id` (including same-UID hijack) or unregistered `tool_hash`.	`SO_PEERCRED` binding hash over (uid, gid, pid, session_id) checked on every request; `tool_hash` must match a kernel-resident manifest entry.
Replay	Ticket reused past its TTL, or after the referenced tool was re-registered.	Tickets carry (`binding_hash`, `binding_epoch`) + TTL; re-registering a tool bumps its catalog epoch, invalidating dependent tickets and forcing session rebind.
Substitution	Manifest edited post-registration, or backend binary swapped behind a live socket.	Kernel-verified manifest SHA-256; first successful probe pins a TOFU `binary_hash` over `/proc/<pid>/exe` (composite for interpreters), re-checked on every invocation.
Escalation	Tool invoked without a valid approval via an `mcpd` logic bug (missed state transition, misrouted approval, ticket-state TOCTOU).	Kernel is the authoritative store for ticket state, so forwarding errors in `mcpd` surface as kernel-side rejection rather than silent admission.
Audit loss	Kill `mcpd` to erase recent call history.	Per-agent `call_log` is a fixed-size kernel ring buffer, exposed read-only via `sysfs`; survives daemon termination.

Table 2: Control-plane threats and defences (in scope). The last row is an observability property rather than a classical attack class.

2.6 Threat Model

The above limitations suggest that the correctness of MCP-style systems depends on a small set of control-plane invariants, including identity binding, approval validity, and consistency between tool semantics and execution. We therefore ground our design in a threat model that focuses on attacks against these invariants in a local-host setting.

The local-adversary model is appropriate for MCP-style deployments for three reasons. First, typical MCP deployments co-locate tool services with the gateway on a single host, so the control plane itself is a local attack surface. Second, `mcpd` exposes no network interface but only a Unix domain socket, so a remote adversary without on-host code execution cannot reach the control plane at all, while one with code execution is indistinguishable from a local unprivileged process; the local model therefore captures both cases without a separate remote treatment. Third, two threats frequently raised in the AI agent setting fall outside this model because they live at other layers, namely prompt injection at the model's reasoning layer and supply-chain compromise of manifests at the filesystem integrity layer; neither is an appropriate target for kernel-level admission control, and both are addressed by complementary defences at those layers, as discussed later in this section. The threat model therefore focuses on what kernel enforcement can actually address: a co-located unprivileged process that can observe, forge, or replay control-plane messages exchanged with the gateway.

Concretely, the adversary model grants the attacker four capabilities: (1) arbitrary code execution under an unprivileged uid distinct from the one under which `mcpd` runs; (2) access to

any Unix domain socket the gateway exposes to unprivileged processes; (3) the ability to craft and replay arbitrary JSON-RPC messages on that socket, including messages observed in prior interactions; and (4) full knowledge of the MCP protocol, tool manifests, and `linux-mcp` implementation, consistent with Kerckhoffs's principle. Correspondingly, the attacker interacts with `linux-mcp` exclusively through the `mcpd` JSON-RPC surface and holds none of the three elevated capabilities that would bypass this interface: no `CAP_NET_ADMIN`, no `ptrace` access to `mcpd`, and no write access to the manifest directory. Each boundary closes a distinct bypass path: `CAP_NET_ADMIN` would let the attacker talk to the kernel module directly via Netlink, skipping `mcpd` entirely; `ptrace` would let the attacker read or modify `mcpd`'s address space and forge control-plane state from within the gateway; and manifest write would poison the kernel registry at the next registration, since manifests are the trusted input from which tool identity is derived.

Two dominant attack categories in AI agent systems fall outside this scope. *Prompt injection* operates at the model's reasoning layer, before any `tool:exec` request is submitted; kernel admission control has no visibility into the planning process [13, 26]. *Supply-chain compromise of manifests* requires file-system-level defences such as Integrity Measurement Architecture or device-mapper verity; `linux-mcp` treats on-disk manifests as trusted input and assumes the adversary lacks write access to `tool-app/manifests/`.

Trust in `mcpd` is structural rather than continuous. The daemon is trusted to perform two translations correctly: at startup, converting manifest contents into SHA-256 hashes and registering them with the kernel; and at each session open, converting the client's `SO_PEERCRED` into a binding hash anchored in the kernel. Once these registrations and bindings are in place, the kernel holds canonical state and re-checks it on every `TOOL_REQUEST` and `APPROVAL_DECIDE`, so the daemon's runtime correctness is no longer load-bearing for individual request validation. This placement defends against (a) daemon logic bugs in request forwarding such as session-id mismatch, missed ticket transitions, or concurrent TOCTOU on ticket state, (b) daemon crashes, since tickets and audit state survive in the kernel, and (c) unprivileged co-located adversaries forging UDS traffic, since session binding is anchored to peer credentials they cannot spoof. It does *not* defend against a `mcpd` whose process memory is directly compromised at any point in its lifetime: a compromised daemon still holds a valid `CAP_NET_ADMIN` handle and can drive the Netlink interface with apparently legitimate calls.

Within this scope, we focus on four classes of control-plane failures: (1) *spoofing* of identity or session context, (2) *replay* of previously valid requests or approval decisions, (3) *substitution* of tool identity or semantics, and (4) *escalation* through bypassing admission checks in the gateway. A fifth property, *post-compromise audit continuity*, is orthogonal to these four: a gateway crash should not erase the record of recent activity. Table 2 summarises the five and the corresponding kernel-anchored defences. Three of those defences go beyond manifest-hash admission alone: trust-on-first-use pinning of the backend binary identity, per-tool catalog-epoch

rebind on manifest change, and a fixed-size kernel call log that survives daemon failure. Their mechanisms are described in Chapter 2.6 and exercised adversarially in Chapter 4.

The trusted computing base is the kernel module, the operator's TOML configuration, and the manifest directory. `mcpd` is trusted to populate the kernel's view of the catalog and to bind sessions at connection time; once populated, the integrity of tickets, tool identity, and audit state does not depend on the daemon staying alive. The supplied systemd unit (Section 3.8) runs `mcpd` as an unprivileged user with only `CAP_NET_ADMIN` and `CAP_SYS_PTRACE`, as defence-in-depth rather than part of the model itself.

Chapter 3: Design and Implementation

The central question driving the design of `linux-mcp` is not how to make the gateway more robust, but which state the gateway should not be solely responsible for holding. `mcpd` retains everything requiring semantic flexibility; `kernel_mcp` owns the state that must survive gateway failure and remain unreachable by unprivileged processes.

3.1 Design Goals

Chapter 1.3 established that MCP-style control planes need enforcement beyond user-space self-policing. `linux-mcp` translates that conclusion into three design commitments.

(1) Kernel-resident admission. Every tool invocation is admitted against kernel-resident state only: the registration of the requesting agent, the presence of the target tool in the registry, and the consistency of its semantic identity with the registered manifest. The admission outcome does not depend on `mcpd`'s in-memory state at the moment of the decision.

(2) No kernel-facing obligations for tool services. Tool authors write against an ordinary user-space IPC interface; the kernel-facing protocol is confined to `mcpd` and described in Section 3.4. This preserves compatibility with existing tool ecosystems and keeps incremental deployment practical.

(3) Kernel-queryable observability. Admission, approval, and completion records are readable through kernel interfaces independently of `mcpd`'s liveness, so a daemon crash does not erase recent control-plane history.

The scope of what these commitments cover, and what they intentionally do not (runtime semantic verification of tool behaviour, a general policy language, the broader attack surface of LLM-based systems), is fixed by the threat model of Chapter 1.3.

3.2 System Architecture

The overall architecture of `linux-mcp`, shown in Figure 1, comprises four runtime components.

`llm-app` is the user-facing client. It queries `mcpd` for the tool catalog, constructs invocation payloads from tool schemas, and submits requests. The client has no direct interaction with the kernel module; all control-plane operations are mediated through `mcpd`.

`mcpd` is the user-space gateway and the semantic front-end of the control plane. It is responsible for loading and validating tool manifests, computing manifest-derived semantic hashes, binding each session to the peer's `SO_PEERCRED`-derived identity and registering that binding with the kernel, exposing catalog APIs (`list_apps`, `list_tools`), communicating with the kernel over Generic Netlink, and forwarding admitted requests to tool services over Unix domain sockets.

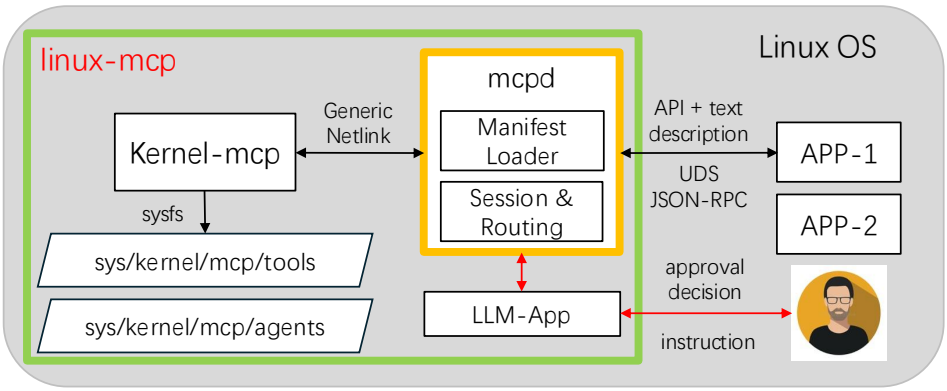


Figure 1: Layered architecture of `linux-mcp`, showing the four runtime components and the wire protocols connecting them.

`mcpd` is the only component that sees both tool semantics and runtime endpoints, so it is the only one that needs changing when manifests or endpoints are reconfigured.

`kernel_mcp` is the durable back-end of the control plane: a loadable kernel module implementing the `KERNEL_MCP` Generic Netlink family. It maintains the tool and agent registries, arbitrates invocation requests, manages approval ticket lifecycles, verifies the session binding hash on every request, pins a trust-on-first-use `binary_hash` for each backend process, and records per-agent invocation history in a fixed-size `call_log` ring buffer. All kernel-held state is exposed read-only through `sysfs` under `/sys/kernel/mcp/`, which remains readable even when `mcpd` is not running. The module does not parse manifests, execute tools, or implement a policy language; its only job is enforcing the admission-control invariants.

`tool-app` is the execution side: a collection of resident user-space services, each bound to a Unix domain socket. When a request is admitted, `mcpd` forwards the payload to the appropriate tool service, which executes in an ordinary user-space environment and returns its result to `mcpd`, which then reports completion back to the kernel. The directory `tool-app/manifests/` is the authoritative source of tool identity: `mcpd` reads manifests from here, hashes them, and registers the resulting digests with the kernel.

The four components communicate over three transport channels. `llm-app` talks to `mcpd` via JSON-RPC over a Unix domain socket at `/tmp/mcpd.sock`; `mcpd` talks to `kernel_mcp` via Generic Netlink on the `KERNEL_MCP` family; and `mcpd` forwards admitted invocations to tool services over per-service Unix domain sockets under `/tmp/linux-mcp-apps/`. Kernel-held state is additionally exposed read-only via `sysfs` under `/sys/kernel/mcp/`, the only channel that does not require a live `mcpd`.

To clarify the separation of responsibilities across layers, Table 3 summarises each component together with its intended scope and explicitly excluded functionality.

Layer	Component	Main responsibility	Intentionally excluded
Client layer	`llm-app` CLI/GUI	Discover tools, plan usage, open sessions, submit requests, handle approvals	Direct tool access, direct kernel communication
Gateway layer	`mcpd`	Load manifests, compute semantic hash, validate payloads, bind sessions, route requests, communicate with kernel	Durable approval and audit state that must survive daemon restart
Kernel layer	`kernel_mcp`	Maintain registries, perform admission decisions, manage approval tickets, verify session bindings, pin `binary_hash`, expose `sysfs` state	JSON parsing, tool execution, LLM-specific logic
Execution layer	Tool services	Execute application logic and return structured results	Any control-plane role (admission, session binding, approval tracking)

Table 3: Concrete layers in `linux-mcp`.

3.3 Control-Plane Workflow

The end-to-end path of a single invocation is:

$$\texttt{llm-app} \rightarrow \texttt{mcpd} \rightarrow \texttt{kernel_mcp} \rightarrow \texttt{mcpd} \rightarrow \texttt{tool-app} \rightarrow \texttt{mcpd} \rightarrow \texttt{llm-app}$$

Figure 2 illustrates this path as a sequence diagram. This path captures one `tool:exec` invocation; a one-time `open_session` handshake precedes the first such invocation on a given session and is described below. A request originates in `llm-app`, which first issues a catalog query that `mcpd` satisfies from its manifest-derived in-memory state. The kernel occupies this path not as a passive observer but as the active arbitration point: `mcpd` cannot forward a request to a tool without first obtaining an `ALLOW` decision from the kernel, and it cannot close an invocation without reporting `TOOL_COMPLETE` back through the same channel. Admission and completion are explicit, kernel-visible state transitions; the daemon no longer makes these decisions in private.

Before issuing any `tool:exec`, the client opens a session with an `open_session` call. On receiving this, `mcpd` reads the peer's credentials from the UDS connection via `SO_PEERCRED`, hashes $(\text{uid}, \text{gid}, \text{pid}, \text{session_id})$ into a binding hash, and registers the binding with the kernel. This binding is the anchor the kernel re-verifies on every subsequent request from the session.

The client uses the catalog response to construct a structured invocation payload and submits it as a `tool:exec` call to `mcpd`; the complete message schema is given in Appendix A.1.

Upon receiving the request, `mcpd` performs a series of user-space checks: it resolves the active session, validates the payload against the declared input schema, identifies the target tool, and

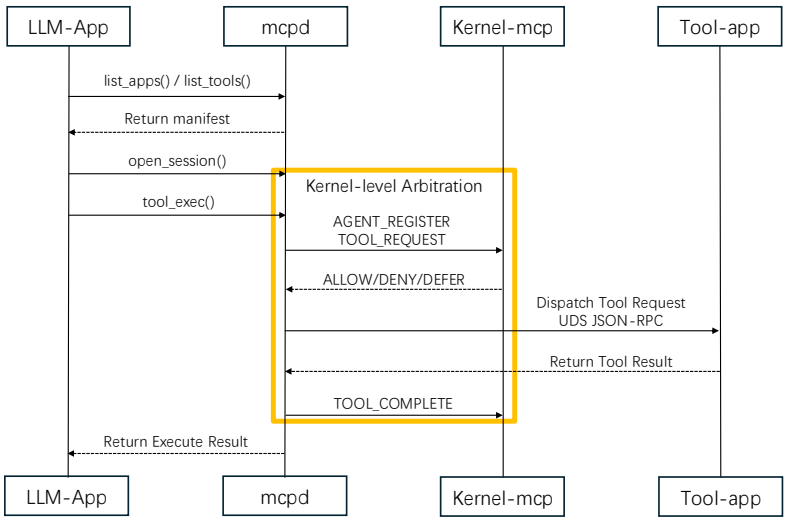


Figure 2: Invocation lifecycle in `linux-mcp`.

retrieves the semantic hash associated with that tool from its manifest-derived state. Only upon successful completion of these checks does `mcpd` issue a `TOOL_REQUEST` to the kernel, supplying the agent identifier, tool identifier, semantic hash, the session binding hash (derived from `SO_PEERCRED` at `open_session`), and the live-probed `binary_hash` of the backend process as parameters.

The kernel module then performs arbitration. It verifies that the requesting agent appears in the agent registry, that the target tool appears in the tool registry, that the semantic hash in the request matches the hash recorded at registration time, that the session binding hash in the request matches the one registered at `open_session`, and that the `binary_hash` — trust-on-first-use-pinned on the first non-empty value — still matches on this invocation. Based on these checks and the tool's declared policy, the kernel returns one of three decisions: `ALLOW`, in which case the request may proceed immediately and the kernel issues an approval ticket that the subsequent `TOOL_COMPLETE` will reference; `DEFER`, in which case the request is held pending an explicit `APPROVAL_DECIDE`; or `DENY`, in which case the request is rejected outright. The detailed semantics of these decisions are described in Figure 3.

Depending on the kernel's decision, `mcpd` either forwards the request to the tool service, transitions the request into a pending state awaiting explicit approval, or terminates the invocation with an error. After the tool service returns — whether with a normal result, an error, a timeout, or a crash caught by `mcpd` — `mcpd` issues `TOOL_COMPLETE` to the kernel, referencing the ticket issued at admission; the kernel records the outcome in the per-agent `call_log` and closes the invocation lifecycle from the control plane's perspective.

This workflow has a concrete observability consequence: the kernel both opens and closes every invocation. An invocation that never reaches `TOOL_COMPLETE`, whether due to daemon failure or tool timeout, persists as an open entry in kernel state and remains inspectable via `sysfs`. This

contrasts with a purely user-space design, in which the lifecycle of a request is only as durable as the daemon's own in-memory representation.

3.4 Wire Protocols

`linux-mcp` uses two transport families: Unix domain sockets for user-space communication, and Generic Netlink for the kernel control-plane path. The user-space UDS variant is operator-configurable: filesystem-path UDS (`uds_rpc`) is the default; abstract-namespace UDS (`uds_abstract`) is also implemented and exercised end-to-end by one demo manifest; a VSOCK variant (`vsock_rpc`) is reserved but not yet wired up. Regardless of variant, the UDS paths between `llm-app` and `mcpd`, and between `mcpd` and each tool service, share a uniform framing: a 4-byte big-endian length prefix followed by a UTF-8-encoded JSON object. A single framing routine therefore suffices on both user-space endpoints. The Netlink path carries TLV-encoded attributes rather than JSON and is described at the end of this subsection.

llm-app to mcpd. `llm-app` sends two categories of message to `mcpd`: system control messages keyed by `"sys"` (covering `list_apps`, `list_tools`, `open_session`, and `approval_reply`) and tool execution requests keyed by `"kind": "tool:exec"`. The two categories are distinguished by the top-level JSON key alone; `tool:exec` dominates in typical workloads, with `sys` messages appearing only at session boundaries. On deferral, the `tool:exec` response carries a `ticket_id` that the client submits with a subsequent `approval_reply` to unblock the request. Complete JSON schemas for all five messages are given in Appendix A.1.

mcpd to tool-app. Once the kernel returns `ALLOW`, `mcpd` forwards the request to the relevant tool service over a per-application UDS connection using a deliberately minimal five-field request / five-field response schema. Consistent with the design goal in Section 3.1 that tool authors should not need to reason about the control plane, the tool service sees only a `(req_id, agent_id, tool_id, operation, payload)` request and returns `(req_id, status, result, error, t_ms)`; it is unaware of sessions, kernel tickets, or the Netlink protocol. The `operation` field is the dispatch string a tool service routes on and is declared per-tool in the manifest; it is distinct from the `tool_id` (numeric, used by the kernel for registry indexing) and from the tool's human-readable `name`. The full field specification is given in Appendix A.1 (Table 10).

mcpd to kernel_mcp (Generic Netlink). The kernel control-plane interface uses the `KERNEL_MCP` Generic Netlink family. Messages carry TLV-encoded attributes, and each message type is registered as a Netlink *command*; all commands except the read-only `LIST_TOOLS` dump require `CAP_NET_ADMIN`. The command set covering registration, invocation, approval, and registry observability is listed in Table 4. The attributes carrying the invariants the kernel actually checks

on the critical TOOL_REQUEST/TOOL_DECISION path are shown in Table 5; the full attribute set and a reference Python framing routine are given in Appendix A.1 (Table 11).

Command	#	Purpose
TOOL_REGISTER	3	Register a tool with its semantic hash and risk flags
AGENT_REGISTER	9	Register an agent with its PID, UID, and binding credentials
TOOL_REQUEST	10	Request admission; kernel replies with TOOL_DECISION
TOOL_DECISION	11	Kernel response: ALLOW (1), DENY (2), DEFER (3)
TOOL_COMPLETE	12	Close the invocation; records execution status and latency
APPROVAL_DECIDE	13	Submit a user approval decision against an open ticket
RESET_TOOLS	14	Clear the kernel-side tool registry (issued on daemon restart)
LIST_TOOLS	8	Dump all registered tools (NLM_F_DUMP mode)

Table 4: KERNEL_MCP Generic Netlink command set. All commands except LIST_TOOLS require CAP_NET_ADMIN.

Attribute	#	Type	Description
TOOL_HASH	19	*string*	64-char hex SHA-256 of manifest fields; drives the Substitution check
BINARY_HASH	33	*string*	SHA-256 of backend's /proc/<pid>/exe (composite with script bytes for interpreter-hosted tools); drives the TOFU check
AGENT_BINDING	28	*u64*	Session binding hash over SO_PEERCRED; drives the Spoofing check
AGENT_EPOCH	29	*u64*	Session opened-at epoch; drives the cross-session replay check
DECISION	17	*u32*	Kernel verdict: 1=ALLOW, 2=DENY, 3=DEFER
TICKET_ID	22	*u64*	Approval ticket handle returned on DEFER and replayed on consume

Table 5: The six KERNEL_MCP attributes on the admission-decision path, each tied to one defence in the Background threat model (Table 2). Routing-level attributes (REQ_ID, TOOL_ID, AGENT_ID, APPROVAL_DECISION) and the per-tool catalog epoch are documented in Appendix A.1.

The BINARY_HASH attribute warrants specific mention because its production is hybrid. mcpd computes the digest in user space: it opens a socket to the backend, resolves the peer PID via SO_PEERCRED, and hashes the bytes of /proc/<pid>/exe at probe time. For interpreter-hosted tools whose manifest declares an entrypoint script, the digest is a composite of the interpreter

executable and the on-disk script bytes, re-read on every probe, so substituting the script between invocations invalidates the pin. The kernel applies trust-on-first-use enforcement to whatever `BINARY_HASH` arrives: the first non-empty value (whether on `TOOL_REGISTER` or a subsequent `TOOL_REQUEST`) locks the slot, moving its `binary_hash_state` from `unpinned` to `live_pinned`; any later `TOOL_REQUEST` whose `BINARY_HASH` fails to match the pinned value is denied with reason `BINARY_MISMATCH`. The kernel never reads `/proc` itself; its role is to store and compare, not to probe.

Tool registration via manifests. Tool services do not self-register; each application is described by a JSON manifest in `tool-app/manifests/` from which `mcpd` derives semantic hashes and issues `TOOL_REGISTER` Netlink commands at startup. Both the `endpoint` and the `transport` fields are excluded from the semantic hash: they describe where the backend is reached, not what it does, so either can change without altering the tool's identity to planners or to the kernel. A separate binding fingerprint over these two fields drives kernel-side rebind when either changes, which clears the TOFU `binary_hash` pin at the new address; the mechanism is detailed in Section 3.5. The complete field specification and a representative manifest are given in Appendix A.1 (Table 12).

3.5 Manifest Semantics and Tool Identity

Defining tool identity at the right granularity is a delicate balance. Binding identity to transport details such as socket paths or process identifiers makes routine operational changes, including redeployments, restarts, and reconfiguration, invalidate registry state even when the tool itself has not changed. If identity is defined too coarsely, however, the system can no longer distinguish a semantically modified tool from one that has merely been redeployed at a different address.

`linux-mcp` resolves this tension by separating semantic identity from transport location. Each tool is described by a JSON manifest whose logical attributes are: tool identifier, tool name, application identifier, application name, risk tags, description, input schema, examples, path semantics, approval policy, and operation; these attributes describe what the tool is intended to do, not where it runs. The `operation` field is the RPC method actually invoked on the backend, so a silent edit retargets execution to a different code path and must change the tool's identity. `mcpd` canonicalises these fields into a stable JSON serialisation and computes its SHA-256 digest, represented as a 64-character hexadecimal string. The full digest is used without truncation: a 32-bit prefix would reach a 50% collision probability at roughly 2^{16} registrations by the birthday bound, and meaningful collision risk appears far earlier—even a few thousand tools put the collision probability well above acceptable thresholds for a primitive the kernel treats as a unique identity.

Two fields are deliberately excluded from the semantic hash: the `endpoint` and the `transport`. Both describe *where* a backend is reached rather than *what* it does, and routine operational moves — switching a service to a new socket path, or between transport variants such as `uds_rpc` and `uds_abstract` — should not cause a tool to look semantically different to planners or to kernel-side hash checks. Instead, `mcpd` tracks these two fields in a separate *binding fingerprint*, also a SHA-256 digest, maintained alongside but distinct from the semantic hash. A change in either field produces a new binding fingerprint, which drives an unregister-then-re-register cycle against the kernel; the cycle clears the kernel's trust-on-first-use `binary_hash` slot on the grounds that the backend reached at the new address may not be the same process. The semantic hash is unaffected by the move, so planners and approval records that reference the tool remain valid.

Any modification to a semantically significant field, including the input schema, risk tags, description, approval policy, path semantics, or `operation`, produces a different semantic digest, which the kernel detects as a mismatch on the next `TOOL_REQUEST`.

This separation is what makes kernel-side admission control meaningful in the first place. The kernel and the daemon can only agree on whether a request is legitimate if they share a stable notion of tool identity, and that notion has to be verifiable on both sides of the privilege boundary. The semantic hash is the concrete artefact that bridges the two: computed in user space from the manifest, but recorded in and checked by the kernel, so a daemon bug or a tampered manifest cannot quietly substitute one tool for another.

3.6 Session Management and Approval Binding

While deferring and resolving a request is mechanically straightforward, binding an approval to the exact session, requester, and operation that triggered it requires explicit ticket-level state and precise matching logic to prevent TOCTOU violations.

Each session is bound to the real peer credentials of its Unix domain socket connection [45]. The daemon takes `uid`, `gid`, `pid`, and a per-session random `session_id`, feeds them into BLAKE2b with an 8-byte output size, and stores the resulting 64-bit digest as the binding hash together with a session epoch. Including `pid` and a random `session_id` means two processes sharing a UID cannot reuse each other's session. The 64-bit output matches the width of the `AGENT_BINDING` Netlink attribute, so `mcpd` re-submits the hash on every `TOOL_REQUEST` and the kernel compares it against the value registered at `open_session`. A `DEFER` response creates an approval ticket tagged with this binding context and a five-minute expiry, overridable per request. When `APPROVAL_DECIDE` arrives, the kernel re-checks the binding before letting the request proceed.

Session-scoped binding plus per-request re-verification rules out approval replay across sessions and keeps the state validated at deferral identical to the state at execution, so no TOCTOU window opens [46, 47].

Catalog-epoch rebind on manifest change. A second form of staleness arises when the catalog changes while a session is open. Without further mechanism, a client could keep invoking a re-registered tool under its pre-change binding. The check is scoped per tool. Each tool records a `registered_at_epoch` at (re-)registration, each session records an `opened_at_epoch` at open time, and a `TOOL_REQUEST` is denied with `catalog_stale_rebind_required` when the targeted tool's `registered_at_epoch` exceeds the session's `opened_at_epoch`. Only the affected tool's re-registration invalidates pending sessions. Editing an unrelated manifest advances the shared `tool_catalog_epoch` counter without breaking other sessions.

Beyond the kernel-side ticket flow, `linux-mcp` supports two orthogonal approval mechanisms driven by different manifest fields and enforced at different layers. The kernel remains the sole arbiter on the control-plane path. The user-space mechanism is a client-side pre-check that either terminates a request before it reaches `mcpd` or lets it proceed unchanged.

(1) Kernel-mediated approval (DEFER ticket flow). The kernel decides between `ALLOW`, `DENY`, and `DEFER` on every `TOOL_REQUEST` based on the tool's `risk_flags`, a bitmask derived from the manifest's `risk_tags` at registration time. Tools tagged `FILESYSTEM_DELETE`, `DEVICE_CONTROL`, `EXTERNAL_NETWORK`, `PRIVILEGED`, or `IRREVERSIBLE` are deferred by default. The five validity checks that distinguish `ALLOW` from `DENY`, and the state machine that governs the resulting tickets, are given in full in the Kernel Module Design section. When a request is deferred, `mcpd` records the pending operation and returns the ticket identifier to the client. After the user approves, `mcpd` first sends `APPROVAL_DECIDE` to the kernel to transition the ticket, then replays the original request with the ticket identifier. The kernel consumes the ticket during the second `TOOL_REQUEST` and returns `ALLOW`.

(2) Client-side local approval (payload-aware prompt). `llm-app` consults each tool's `approval_policy` field before sending `tool:exec` to `mcpd`. Most tools omit the field and incur no local prompt. A tool may declare `approval_policy.user_confirmation` with a condition such as `when = "path_outside_repo"`, in which case the client resolves the payload's declared path fields against the repository root and prompts the user if any of them resolve outside. The demo tool `write_host_text_file` in `04_desktop_app.json` uses exactly this to require confirmation before writing to the host filesystem outside the project directory. A declined local prompt terminates the request in user space. No `tool:exec` is sent to `mcpd` and no kernel ticket is created.

Relationship between the two mechanisms. They are complementary, not mutually exclusive. Kernel `DEFER` is *payload-agnostic*; once a tool carries a high-risk flag, every invocation is gated regardless of arguments. The local prompt is *payload-aware* and only gates requests whose specific arguments meet the declared condition. A single tool can trigger both, in which case execution proceeds only after the local prompt is approved, an `APPROVAL_DECIDE` succeeds, and the ticket-bearing second `TOOL_REQUEST` is admitted.

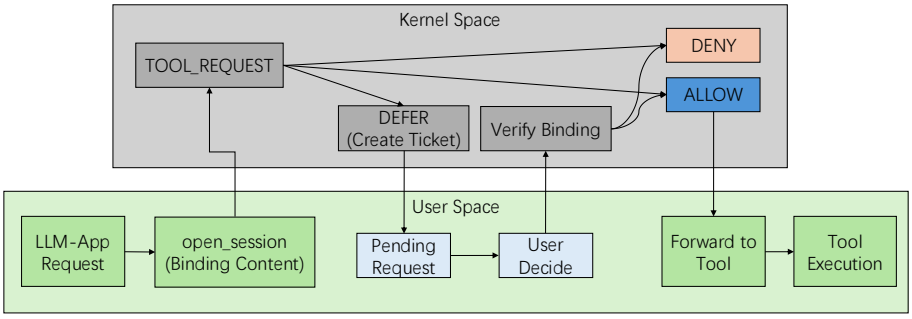


Figure 3: Approval-ticket flow in `linux-mcp`: the `ALLOW`/`DENY`/`DEFER` state machine enforced by the kernel module.

Figure 3 illustrates the kernel-side state machine. Approval resolution always happens *before* tool execution, so no deferred request bypasses the approval gate and every deferred execution consumes an explicit ticket. The kernel need not understand the full meaning of a request. It verifies that the binding context is preserved, the ticket is valid and unconsumed, and the `DEFER` → `ALLOW` transition is explicitly authorised [48].

3.7 Kernel Module Design

`kernel_mcp` is implemented as a loadable kernel module exposing the `KERNEL_MCP` Generic Netlink family. The command set covers registration, invocation, approval, and registry observability: `TOOL_REGISTER`, `AGENT_REGISTER`, `TOOL_REQUEST`, `TOOL_DECISION`, `TOOL_COMPLETE`, `APPROVAL_DECIDE`, `RESET_TOOLS`, and `LIST_TOOLS`. Generic Netlink was chosen over `ioctl` or a character device because it provides structured TLV attributes with kernel-side schema validation, native support for dump-style enumeration (used by `LIST_TOOLS`), and per-command `GENL_ADMIN_PERM` gating that integrates cleanly with Linux capabilities [49].

Internally, the module maintains two registries. The tool registry maps tool identifiers to their manifest-derived semantic hashes and associated metadata. Each entry also carries risk flags, a per-tool `registered_at_epoch`, and the TOFU `binary_hash` slot. The agent registry records agents permitted to submit requests through the control plane, together with the `binding_hash`, `binding_epoch`, and `opened_at_epoch` captured at `open_session`. Approval tickets are heap-allocated kernel objects with an explicit state machine. The normal path runs `PENDING` → `APPROVED` → `CONSUMED`. A reject takes `PENDING` to `DENIED`. A post-approval revoke takes `APPROVED` to `DENIED`. `EXPIRED` is implicit and is represented by removal from the ticket table rather than as a stored state.

The arbitration logic is deliberately narrow. A `TOOL_REQUEST` is admitted only if five conditions all hold. The submitting agent must appear in the agent registry. The target tool must appear in

the tool registry. The semantic hash supplied by `mcpd` must match the hash recorded at registration. The session `binding_hash` in the request must match the one registered at `open_session`. If the TOFU slot is already pinned, the `binary_hash` in the request must match the pinned value. Any failing check produces `DENY` with a specific reason code. Once the checks pass, the kernel consults the tool's `risk_flags`. A tool carrying `FILESYSTEM_DELETE`, `DEVICE_CONTROL`, `EXTERNAL_NETWORK`, `PRIVILEGED`, or `IRREVERSIBLE` is deferred and receives a fresh approval ticket; everything else gets `ALLOW`.

Trust-on-first-use backend binary pinning. Semantic-hash admission guards what a tool claims to be, not which binary actually serves the socket. The kernel therefore keeps a per-tool `binary_hash` slot that `mcpd` fills on a live probe. `mcpd` reads the backend's peer PID via `SO_PEERCRED` and hashes `/proc/<pid>/exe`. For interpreter-hosted tools the digest combines the interpreter with the on-disk script, so a script swap is still caught. The slot locks on the first non-empty digest, which usually arrives on `TOOL_REGISTER` but may arrive on the first successful `TOOL_REQUEST` if the backend comes up later. Any subsequent `TOOL_REQUEST` whose digest does not match is rejected with reason `BINARY_MISMATCH`. A `binary_hash_state` field distinguishes `unpinned` from `live_pinned`, since both previously read as an empty string.

Kernel-resident call log. Each agent carries a fixed-size 32-slot circular buffer of invocation records. A record is a flat struct holding a monotonic sequence number, a nanosecond timestamp, the request and tool ids, a status (one of `OK`, `ERR`, `DENY`, `DEFER`), execution latency, a tool status code similar to a process exit code, 8-byte payload and response hash prefixes, and up to 48 leading bytes of any error output. Completed invocations log from `cmd_tool_complete`; denied or deferred ones log from `cmd_tool_request`, so nothing escapes the audit buffer. The kernel parses no JSON. `mcpd` precomputes the hashes and passes them as fixed-width attributes. The buffer is exposed through `sysfs` as a binary attribute and stays readable after a `mcpd` crash. This is crash-visible audit rather than tamper-evident provenance, since the buffer is bounded and the hashes are 8-byte prefixes.

Sysfs surface. Each tool directory under `/sys/kernel/mcp/tools/` exposes `name`, semantic `hash`, `binary_hash`, `binary_hash_state`, and `registered_at_epoch`. The global `tool_catalog_epoch` sits at the top of the tree. Per-agent entries under `/sys/kernel/mcp/agents/` expose per-state counters (`allow_count`, `deny_count`, `defer_count`, `completed_ok_count`, `completed_err_count`, `last_exec_ms`), a human-readable `last_reason`, and the `call_log` above with its head and count indices. An operator can read what the kernel regards as authoritative without relying on daemon-side logs.

3.8 Daemon Implementation

The kernel enforces the critical state transitions, but `mcpd` does most of the actual work. Its code splits across four concerns. `manifest_loader.py` handles manifest loading and semantic hashing. `session_store.py` manages session storage and binding context. `netlink_client.py` carries Netlink traffic to the kernel. `server.py` dispatches requests and runs the backend probe. Sections 3.2, 3.5, 3.4, and the session discussion above cover each area in detail. Two daemon-specific behaviours deserve separate treatment below. One is how `mcpd` keeps the kernel registry consistent across startups and manifest edits. The other is how it closes the TOFU `binary_hash` loop when the backend is unreachable.

Kernel state reconciliation. The manifest directory is the source of truth, but kernel registry state persists across `mcpd` restarts. A cold start or a manifest edit must neither drift silently from the on-disk manifests nor invalidate existing sessions unnecessarily. On every reload, the daemon computes a per-tool diff between the previous and new registries and collapses it into one of five reconcile actions. `KEEP` applies when both fingerprints match and makes no kernel call. `ADD` and `REMOVE` handle tools that appear or disappear between reloads. `RE_REGISTER_SEMANTIC` covers a changed semantic hash on an unchanged backend, and the in-place re-register preserves the kernel-side TOFU `binary_hash` slot. `RE_REGISTER_BINDING_MOVE` fires when `transport` or `endpoint` changed, which means the backend may now be a different process. The daemon first unregisters to clear the TOFU slot, then re-registers so the next probe pins a fresh digest. Two fingerprints from Section 3.5 drive the decision. A change in `manifest_hash` alone selects `RE_REGISTER_SEMANTIC`; a change in `binding_fingerprint` escalates to `RE_REGISTER_BINDING_MOVE`.

Probe-failure fail-closed. The TOFU `binary_hash` mechanism only works if the daemon never quietly substitutes an empty digest for a failed probe. If `mcpd` cannot reach the backend, cannot read `/proc/<pid>/exe`, or does not recognise the transport, the probe returns a non-live result and the exec path refuses the invocation. No empty `BINARY_HASH` flows upward. This closes two silent failure modes. A fresh probe that cannot read the live binary will not reset a pinned slot and let a swapped backend start clean. A cold registration before the backend comes up will not install a blanket empty pin that a later swap could inherit. Audit records for probe refusals carry `source=probe`, which separates them from kernel-side denials (`source=arb`) and client-side declines (`source=user_decline`).

Netlink client. `mcpd` keeps a single long-lived Netlink socket to `kernel_mcp` rather than re-opening one per request. Socket setup adds measurable overhead on an otherwise microsecond-scale path, and a persistent connection simplifies the ordering of `APPROVAL_DECIDE` against

its eventual TOOL_REQUEST replay. Optional per-stage latency counters (session lookup, kernel arbitration, tool execution, end-to-end) are available on this path for the overhead analysis in Chapter 3.11.

Deployment and fail-closed configuration. mcpd needs CAP_NET_ADMIN for Netlink administration and CAP_SYS_PTRACE for reading /proc/<pid>/exe across uid boundaries during the backend probe. The supplied systemd unit runs the daemon as a dedicated unprivileged mcpd user and grants both capabilities via AmbientCapabilities. The same two capabilities also appear in CapabilityBoundingSet, so an execve-launched child cannot regain anything beyond what mcpd itself uses. ProtectSystem=strict with a ReadWritePaths allowlist of /tmp and /tmp/linux-mcp-apps renders the rest of the filesystem, including the manifest directory, read-only to the daemon, with no explicit bind mount required.

Two further directives harden the process itself. NoNewPrivileges=true blocks any child execve from picking up capabilities mcpd lacks. MemoryDenyWriteExecute=true prevents a memory-corruption exploit from mapping writable executable pages, which blunts an attack class the threat model otherwise excludes. Several standard directives shrink the blast radius if mcpd is compromised. ProtectKernelTunables, ProtectKernelModules, and ProtectControlGroups block kernel-state probing. RestrictSUIDSGID and LockPersonality cut off SUID and personality-flag tricks. PrivateDevices strips the /dev access mcpd never uses.

A separate question is which UIDs the daemon accepts as tool backends. mcpd checks allowed_backend_uids via SO_PEERCRED on every probe and exec dial. An unprivileged mcpd defaults to its own UID. A privileged mcpd has no safe default. Trusting only UID 0 rejects every realistic non-root backend and leaves binary_hash unpinned. Trusting $SUDO_UID folds the correctness of the launcher script into the daemon's trust boundary. mcpd therefore refuses to start unless the list is declared in the TOML, or the operator explicitly opts in via LINUX_MCP_TRUST_SUDO_UID. The supplied systemd unit sets LINUX_MCP_CONFIG to a TOML path and leaves LINUX_MCP_TRUST_SUDO_UID unset. Production deployments therefore name backend UIDs in configuration rather than inherit them from the launch environment.

3.9 State Reconciliation and Recovery

linux-mcp treats a daemon restart as a recovery problem rather than a process-management detail. The three layers of state it keeps sit on different sides of the crash boundary. The kernel holds the tool registry, approval tickets, the tool_catalog_epoch counter, and each agent's call_log. All of that survives a mcpd crash. The manifest set on disk is the authoritative source of tool identity and is unaffected by restart. Open sessions, peer-credential bindings, and pending request maps live only in mcpd's memory and are discarded when the process exits.

The cold-start procedure follows directly from this partition. At startup, mcpd issues RESET_

TOOLS to clear the kernel tool registry and then re-registers each manifest tool unconditionally, which guarantees the kernel and the on-disk manifest set agree before any client is accepted. RESET_TOOLS alone does not bump the catalog epoch; the per-tool register calls that follow do. A surviving session holding a pre-restart opened_at_epoch will therefore receive catalog_stale_rebind_required on its next request against any re-registered tool. The per-tool diff-based reconciliation described in Section 3.8 applies to later manifest edits, not to this cold-start path.

This split preserves audit and admission state without pretending that a crashed daemon can safely continue pre-existing sessions. Approval tickets and registry history stay inspectable through sysfs across the restart, so an operator can still read what recent activity occurred while mcpd was down or unresponsive. Any surviving ticket whose referenced session no longer has a live mcpd-side binding simply expires at its TTL. Clients reopen any session that pre-dates the restart on their first post-restart request. The client library handles this rebind automatically when it sees the catalog_stale_rebind_required reason.

A kernel module reload sits outside this recovery model. Unloading kernel_mcp drops all kernel-held state (registry, tickets, epochs, call logs), so the operator must treat the next start as a fresh system. mcpd re-registers from manifests as usual, but any historical audit kept only in the now-gone kernel buffer is lost.

3.10 Rationale for the Kernel–User-Space Split

Three kernel-interface designs were considered before settling on Generic Netlink.

The first candidate was eBPF. It looked lightweight, needed no full kernel module, and is increasingly the standard way to extend kernel behaviour. An early design review exposed two obstacles rooted in eBPF LSM's hook-attachment model. eBPF LSM hooks attach to kernel operations such as security_socket_recvmsg and security_file_open, not to parsed JSON-RPC messages. An in-kernel enforcer would therefore re-parse MCP framing from raw socket bytes on every hook invocation before it could see a tool_hash or a ticket_id. That parsing also has to fit under the eBPF verifier's bounded-loop and instruction-count constraints while handling variable-length strings and nested JSON objects. State management itself is tractable in eBPF; bpf_timer and pinned maps are adequate for approval tickets and expiry. Reaching that state, however, means rebuilding the application-layer protocol state machine inside verifier-constrained programs. That cost is what made eBPF unattractive for this particular control plane.

LSM hooks were the second candidate. They fire at the syscall boundary, which is the wrong layer. Tool identity in this system is a SHA-256 hash of a JSON manifest, not a file path or a socket operation. No syscall naturally sees a tool hash alongside a request; correlating the two would inject application-layer semantics into a mechanism designed for OS-object-level policy.

The mismatch is fundamental, not a matter of implementation effort.

A capability-based design in the style of Capsicum was also considered. It would have required tool services to interact with a kernel-facing API to obtain and present capabilities, which breaks the goal that tool authors need only write a JSON manifest with no kernel-facing obligations.

Generic Netlink avoids all three problems. It supports structured TLV messages, dump-style enumeration for registry introspection, and arbitrary internal state management in a loadable module. The cost is that a full kernel module is needed where a policy hook or a BPF program would have sufficed for simpler designs. For a prototype targeting a narrow set of admission invariants this is an acceptable trade-off, and the module's role stayed scoped enough that the added complexity remained manageable.

3.11 Simplifications and Optimisation Opportunities

The prototype is scoped for control-plane correctness rather than peak throughput. Several choices along the way combine genuine scalability limits with intentional simplifications, and each is worth naming so it is neither mistaken for a fundamental limit nor inherited uncritically.

A single global lock protects the kernel registries. This is adequate at the evaluated catalog sizes but becomes a scalability concern under higher concurrency. Replacing it (for example with RCU-protected reads on the request path) is the most mechanical of the items listed here.

Cold-start reconciliation is unconditional. Every launch pays a `RESET_TOOLS` plus a full `TOOL_REGISTER` sweep, even when manifests are unchanged. This is correctness-first, as discussed in Section 3.9; the daemon has no trustworthy way to prove that its in-memory view matches the kernel's without the rebuild. An optimisation that skips the sweep when a manifest-set signature is unchanged would cut startup cost at the price of more elaborate startup logic.

Hot reload is request-driven rather than filesystem-driven. `mcpd` recomputes a manifest-set signature on the `list_apps`, `list_tools`, and `tool:exec` paths and refreshes the kernel registry when the signature changes, so a manifest edit takes effect on the next request rather than the instant the file is written. The request-driven variant avoids an `inotify` dependency and the race between a partially-written manifest and a concurrent request, at the cost of a bounded staleness window between edit and uptake.

The kernel performs no per-agent rate limiting; throttling is left to `mcpd`. This keeps the kernel module scope narrow (no policy language, no token buckets) and concentrates policy decisions in the layer that already tracks session context. One consequence is that the kernel cannot defend against a compromised `mcpd` issuing a flood of well-formed `TOOL_REQUEST`s, a limitation consistent with the time-partitioned trust model of Section 2.6.

Section 5.3 discusses resolution directions for each.

Chapter 4: Results and Discussion

`linux-mcp` is evaluated along four dimensions: control-plane latency and its breakdown across invocation stages, throughput under concurrent load, resistance to a representative set of control-plane attacks, and system behaviour under daemon failure and restart. The central claim under test is that relocating admission-control state into the kernel strengthens enforcement without making the tool invocation path prohibitively expensive. Two baselines isolate this effect from user-space hardening alone: a conventional user-space mediator (**userspace**), in which registration, admission, and approval all reside in the daemon, and a hardened variant (**seccomp**) that augments the same control path with stricter syscall filtering, additional validation, and audit logging.

The prototype under test comprises 43 tools across 14 applications; all three systems operate over this same tool deployment throughout the evaluation. All experiments were conducted on a VMware guest (Apple aarch64 host, 4 vCPU, 8 GB RAM) in a headless configuration with no concurrent user-space workloads. Latency results are reported over $n = 30$ independent runs of 2000 requests each, giving a standard error of the mean small enough to support Welch's t-test comparisons at $\alpha = 0.05$ while remaining within the hypervisor timing noise of a virtualised guest. Throughput and the category-level security evaluation are drawn from the same run, using the harness's built-in concurrency sweep and ten repeats of each attack case. All reported p-values are two-sided Welch's t-tests [50] comparing per-repetition mean latency against the `userspace` baseline.

4.1 Latency and Control-Plane Overhead

Table 6 and Figure 4 report mean end-to-end latency for each system at three payload sizes. The 100 B and 10 KB points correspond to typical MCP tool invocations (short JSON commands and small structured responses), while the 1 MB point is a contrived stress case rather than a representative workload: it is used solely to probe how the control plane behaves when the payload dominates the cost of a request, and is not intended to reflect everyday tool traffic. At 100 B, the kernel path and the user-space baseline are within measurement noise: both at 0.854 ms ($p = 0.984$). At 10 KB the kernel path is nominally slower (0.899 ms versus 0.863 ms, +4.2%), but the difference does not reach significance ($p = 0.097$), a sub-millisecond effect lost in the timing noise of a VMware guest at $n = 30$.

The 1 MB result is the one definitive number in the table. The kernel path runs at 7.487 ms, compared to 7.748 ms for plain user-space (a 3.4% reduction, $p = 0.00027$) and 9.843 ms for the hardened variant. This is counter-intuitive: adding a privilege boundary is usually expected to cost latency, not return it. Figure 5 shows the same data as a ratio against the plain `userspace` baseline, with the y-axis zoomed around the parity line so that the sub-4% effects at 100 B and

Table 6: Mean end-to-end latency (ms) by payload size ($n = 30$ runs $\times$ 2000 requests). Bold values indicate the lowest latency per row. p-values are two-sided Welch's t-tests against the `userspace` baseline; ^{ns} = not significant ($p > 0.05$).

	Mean latency (ms)			**Welch's p-value**	
Payload	`userspace`	`seccomp`	`kernel`	vs. `seccomp`	vs. `kernel`
100 B	**0.854**	0.867	**0.854**	0.475^{ns}	0.984^{ns}
10 KB	0.863	0.922	0.899	0.000206	0.097^{ns}
1 MB	7.748	9.843	**7.487**	$<10^{-4}$	0.00027

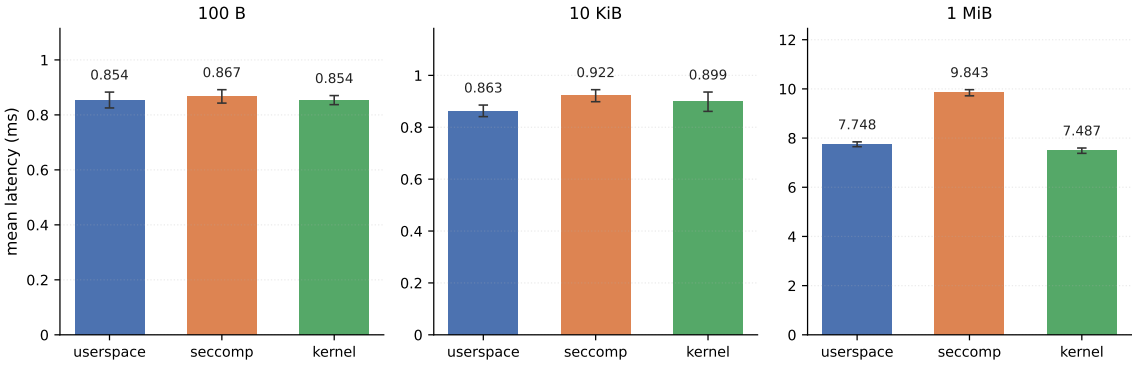


Figure 4: Mean end-to-end latency by payload size for the three systems. Each subplot auto-scales its own y-axis so that the sub-millisecond differences at 100 B and 10 KB are not flattened by the much larger absolute values at 1 MB. Error bars show 95% confidence intervals across $n = 30$ independent runs; significance markers are Welch's t-test against the `userspace` baseline.

10 KB are visible in the same frame as the larger effect at 1 MB; read that way, `seccomp` sits above parity at every payload size while the kernel path crosses below parity only at 1 MB. The explanation has two parts. The seccomp penalty at large payloads is the accumulating cost of BPF per-syscall filtering across the heavier socket traffic that a megabyte payload forces through, and the kernel path does not pay that cost at all. The modest speedup against plain user-space, we believe, reflects early-reject behaviour: arbitration happens before the payload is copied through the tool-forwarding socket, so a rejected request avoids a copy the user-space baseline would still have to perform between admission and forwarding.

The p_{95} picture, shown in Figure 6, matches the means. At 100 B and 10 KB the three systems are statistically indistinguishable ($p = 0.51$ and $p = 0.16$). At 1 MB the kernel p_{95} is 8.422 ms, against 8.716 ms for user-space ($p = 0.049$) and 10.903 ms for seccomp. The tail does not widen under the enforcement boundary.

Stage-level breakdown in Figure 7 confirms that kernel arbitration is a minor contributor at all payload sizes. Mean arbitration time is 0.028 ms at 100 B and 10 KB, and 0.038 ms at 1 MB. At 1 MB, tool execution accounts for 89.3% of total end-to-end time on the kernel path (89.9% on the user-space path); the entire control-plane machinery (Generic Netlink round-trip, session

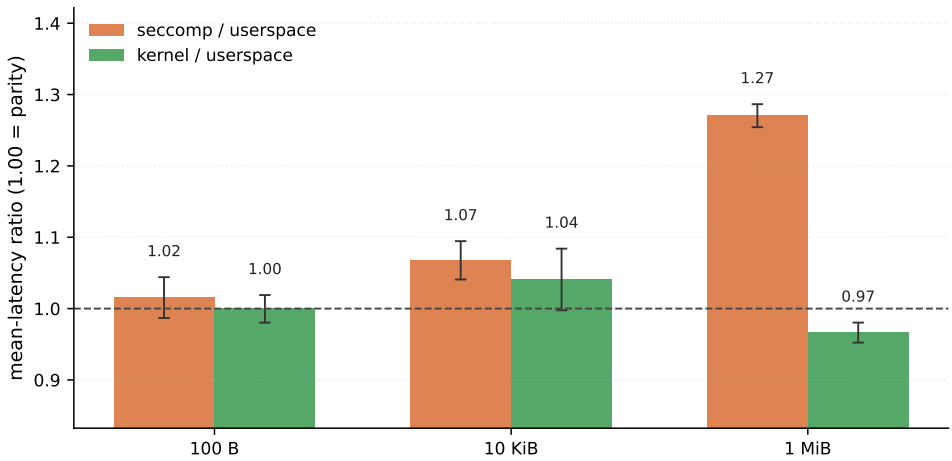


Figure 5: Mean-latency ratio of each hardened variant to the plain `userspace` baseline, with 95% confidence intervals propagated from the 30-run per-repetition means. The y-axis is zoomed around the parity line (ratio = 1.00) to make the sub-4% effects at 100 B and 10 KB visible alongside the larger effect at 1 MB. The `seccomp` path is significantly slower at every payload size; the `kernel` path is statistically indistinguishable from `userspace` at small and medium payloads and measurably faster at 1 MB.

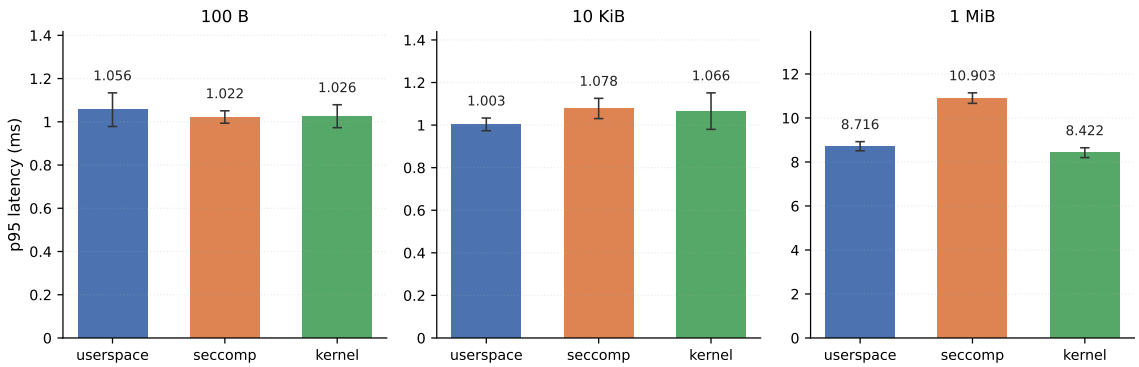


Figure 6: p_{95} latency by payload size for the three systems ($n = 30$ runs of 2000 requests each). Error bars show 95% confidence intervals. At 100 B and 10 KB the kernel path is statistically indistinguishable from the userspace baseline; at 1 MB the kernel p_{95} (8.422 ms) is lower than both userspace (8.716 ms, $p = 0.049$) and seccomp (10.903 ms), confirming that the enforcement boundary does not widen the tail under the tested workload.

lookup, and arbitration) accounts for the remaining 10.7%, with the arbitration contribution within that fraction less than four hundredths of a millisecond.

A supplementary microbenchmark isolates the Generic Netlink layer using a minimum-cost `KMCP_CMD_NOOP` handler added to the kernel module. The NOOP path returns immediately after Generic Netlink demultiplexing, without touching any tool or agent registry, and therefore measures the irreducible floor of a round-trip against which all per-stage arbitration costs can be anchored. Figure 8 reports the medians on a single axis: the NOOP floor sits at 3.125 μs, the full `TOOL_REQUEST` path measured at a realistic registry scale of 1024 tools and 64 agents sits at 7.459 μs, and the `skip_lookups` and `ticket_trigger` bars bracket the per-stage contributions

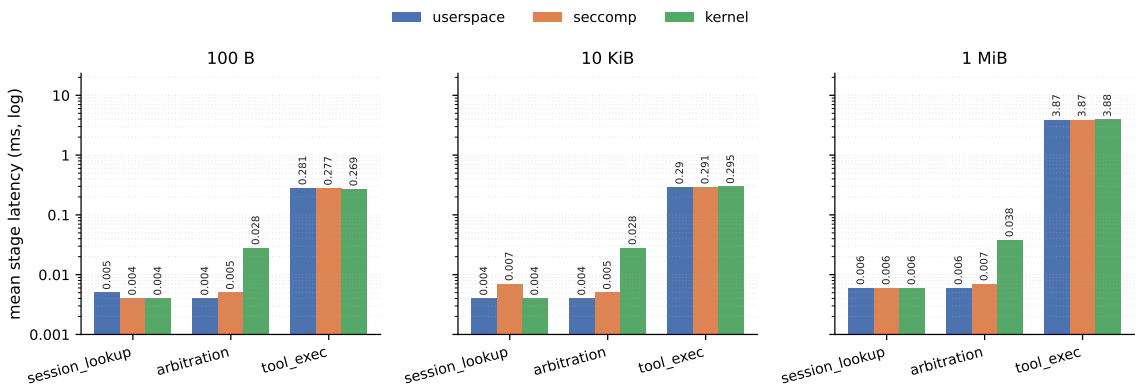


Figure 7: Invocation-stage breakdown (session lookup, arbitration, tool execution) for each system at all three payload sizes, on a log y-axis. The log scale makes session_lookup (around 4 µs) visible alongside tool_exec (around 4 ms). The `kernel` path spends roughly one order of magnitude more time in arbitration than the user-space baselines, but that extra cost is well under 0.04 ms and is invisible relative to the tool-execution component.

that the next paragraph decomposes. The total kernel arbitration cost is therefore 4.334 μs above the noise floor, which is roughly one order of magnitude smaller than the sub-millisecond effects visible in the end-to-end measurements; the dominant cost on the kernel path originates in user-space components (session management, payload serialisation, and socket I/O) rather than in the kernel module itself.

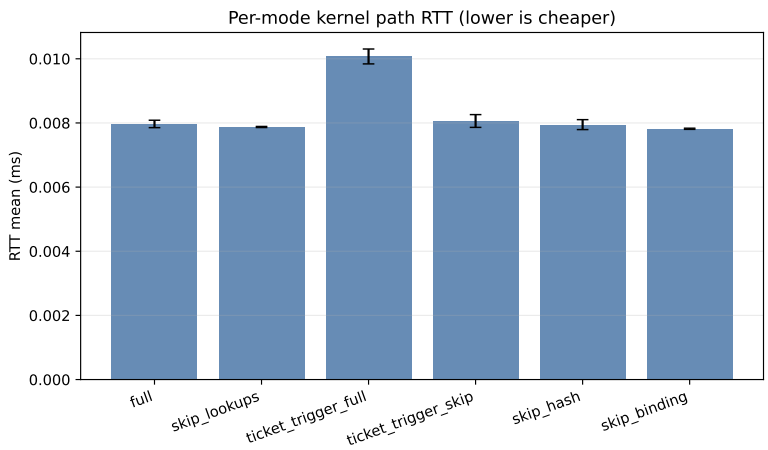


Figure 8: Kernel path ablation at realistic registry scale ($N = 1024$ tools, $M = 64$ agents). The leftmost bar is the measured noise floor from the `KMCP_CMD_NOOP` handler, which establishes the irreducible Generic Netlink round-trip cost. The `full` bar is the complete `TOOL_REQUEST` arbitration path; `skip_lookups` disables the tool and agent registry walks. The two `ticket_trigger` bars exercise a high-risk tool whose execution is gated by the approval-ticket machinery, with and without the approval branch bypassed. All error bars are 95% bootstrap CIs across $n = 10$ repetitions.

To decompose the 4.334 μs arbitration cost into its constituent stages, a paired ablation was performed against compile-time experiment flags that selectively bypass individual kernel code

paths (`SKIP_LOOKUPS`, `SKIP_HASH`, `SKIP_BINDING`, `SKIP_TICKET`). Each stage delta is measured by alternating the baseline and bypass modes request-by-request within a single benchmark phase, so the two samples in every pair are collected microseconds apart and share the same scheduler state and cache residency. Hypervisor-level drift that contaminates naive sequential-mean comparisons cancels out inside the paired difference, and bootstrap CIs on the delta vector are correspondingly tighter.

Table 7 and Figure 9 together report the two stages whose guard bodies are actually executed by the benchmark workload, plus two methodology-check rows that document the path-identity caveat. The `registry+agent_lookup` stage (two mutex pairs, an `xa_load` against a 1024-entry xarray, and a 64-entry hashtable walk) contributes 0.109 μs per request. The `approval_ticket_body` stage (consume plus issue, including an `approval_lock` acquisition and hashtable insert) contributes 2.012 μs per request and is only exercised when the tool is high-risk and the request arrives without a pre-authorised ticket. Under benign credentials the `hash` and `binding` guard bodies are dead code (short-circuited by the workload state before the experiment flag is consulted), and their paired deltas correctly return 0± noise with $p_{\mathrm{BH}} > 0.5$, confirming that the paired method introduces no systematic bias on identical code paths.

Table 7: Paired per-stage ablation deltas. Each row alternates a baseline mode and a bypass mode request-by-request inside a single phase; the Δ column is the mean of $\mathrm{baseline}_i - \mathrm{bypass}_i$ across all pairs. BH-corrected paired t-tests are reported across all four comparisons.

Stage	n_{pairs}	Δ_{avg} (μs)	**95% CI** (μs)	t_{paired}	p_{BH}
registry+agent_lookup	100 000	+0.109	[+0.087, +0.131]	+10.08	$< 10^{-4}$
approval_ticket_body	1 000	+2.012	[+1.760, +2.257]	+15.99	$< 10^{-4}$
Methodology consistency checks (dead guard bodies under benign workload)					
hash_guard_body	100 000	+0.163	[−0.153, +0.541]	+0.88	0.507
binding_guard_body	100 000	−0.007	[−0.027, +0.013]	−0.65	0.513

Subtracting the 0.109 μs registry+lookup cost from the 4.334 μs total leaves roughly 4.2 μs unattributed in the benign run (the approval-ticket body is not exercised along this path, so its cost does not enter the sum). The remainder is reply-skb construction: the five `nla_put_*` calls and the `genlmsg_reply` in `kernel_mcp_reply_tool_decision`, which the experiment flags do not decompose further.

One caveat about the `approval_ticket_body` figure deserves explicit mention. Every ticket issuance calls `kernel_mcp_purge_expired_tickets_locked()`, which iterates the full 256-bucket approval hashtable. Under 300 s TTL, no ticket expires during a one-minute microbenchmark, so the purge scan cost grows linearly with the number of tickets the benchmark has already issued. Moving the expiry scan to the existing periodic `kernel_mcp_ticket_cleanup_timer` would eliminate this $O(n)$ component entirely. This is flagged as a simple future-work opti-

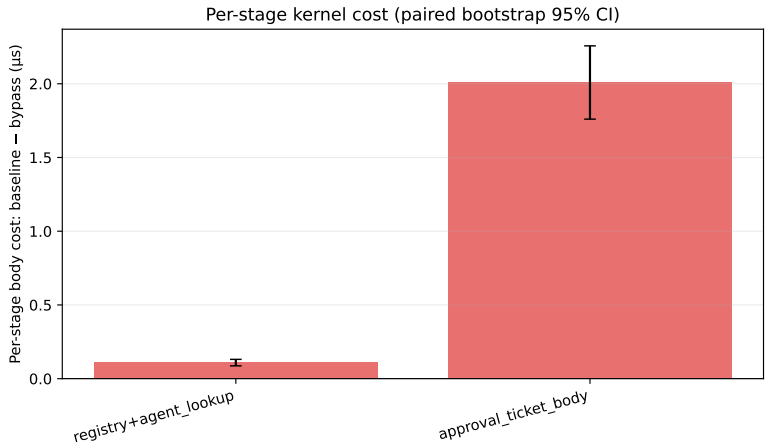


Figure 9: Paired per-stage deltas across the four ablation comparisons. The first two bars are the meaningful stage costs (registry+agent lookup and the approval-ticket body); the last two are methodology consistency checks against dead guard bodies and are expected to fall on zero.

misation, not a characteristic of the fast path in production use where `mcpd` consumes tickets promptly and the hashtable stays small.

4.2 Throughput and Scalability

Throughput was measured across agent counts $\{1, 5, 10, 20, 50\}$ and concurrency levels $\{1, 10, 50, 100\}$ to check whether the kernel-assisted control plane becomes a bottleneck once multiple requests are in flight simultaneously. Table 8 gives the per-cell peak throughput at one agent; the complete sweep across agent counts produces the same shape and is summarised in prose below.

Table 8: Peak throughput (RPS) at one agent, across concurrency levels ($n = 5$ repetitions per cell). The `userspace` column is the highest in every row; the `kernel` path sits roughly 10% below it at every concurrency level, without collapsing.

Concurrency	`userspace`	`seccomp`	`kernel`
1	1117.5	1088.5	1071.1
10	1146.0	1036.9	1009.4
50	1215.6	1123.5	1096.2
100	1213.4	1072.5	1103.7

Plain user-space achieves the highest peak throughput in every row, ranging from 1090 to 1219 RPS across the full grid. The kernel path ranges from 1003 to 1137 RPS, and the seccomp path from 1020 to 1200 RPS. At a representative operating point of one agent and concurrency 50, the three systems sit at 1215.6, 1123.5, and 1096.2 RPS respectively. The kernel path is roughly 10% slower than the user-space peak, which is consistent with the additional Generic Netlink round-trip and with the serialisation of admission decisions through a single

kernel module.

These numbers come from an earlier $n = 5$ pilot run. The $n = 30$ study that drives the latency table was deliberately scoped to latency alone and did not repeat the concurrency sweep, so the throughput figures carry wider uncertainty than the latency figures and should be read as evidence of practical viability rather than as a precise ceiling. Within the tested range the kernel path holds a stable $\sim$10% gap without collapsing, which is enough to support the viability claim. A follow-up sustained-overload run was attempted at concurrencies of 200, 400, and 800, but the `SOMAXCONN` accept queue saturated before the control plane itself became the bottleneck, so those cells produced error-dominated latency distributions rather than usable p99 curves and are not reported here.

4.3 Registry Scaling

A separate experiment characterises how the kernel-side tool registry behaves as the catalogue grows beyond the 43-tool deployment used elsewhere in the evaluation. N synthetic tools (with $N \in \{8, 16, 32, \ldots, 16384\}$) are bulk-registered into the kernel, and both the register path and the steady-state lookup path are measured. The register path is modelled as a one-time setup cost plus a per-tool steady-state cost:

$$\text{total_ms}(N) = a + b \cdot N,$$

fitted across the 12 values of N by ordinary least squares.

The fit produces $a = -0.123$ ms, $b = 6.011$ μs per tool, and $R^2 = 0.9996$. The asymptotic registration throughput is therefore $1/b \approx 166000$ tools/s; the 43-tool catalogue used in the main evaluation registers in approximately 0.26 ms of wall-clock time, and even a 16384-tool deployment completes its one-shot registration in under 100 ms. The per-tool cost curve (Figure 10) approaches the asymptote b from above with no inflection point — there is no scaling degradation at large N, and the apparent variation at small N is the intercept a amortising away.

The steady-state lookup path is measured separately by drawing a random `tool_id` for each request against the pre-populated registry; the log-linear regression lookup_ms $\sim a + b \cdot \log_2 N$ produces a slope of 2.9×10^{-5} ms per doubling with a bootstrap 95% CI of $[-2 \times 10^{-6}, +6.1 \times 10^{-5}]$ that contains zero ($R^2 = 0.26$). The near-zero slope and the CI overlapping zero are consistent with the expected $O(1)$ behaviour of the underlying xarray data structure, and inconsistent with any balanced-tree implementation where slope would grow with $\log_2 N$ at high R^2. Figure 11 shows the lookup latency distribution across the full range of N. Mean lookup latency stays within the 7.1–7.7 μs band across three orders of magnitude of registry size.

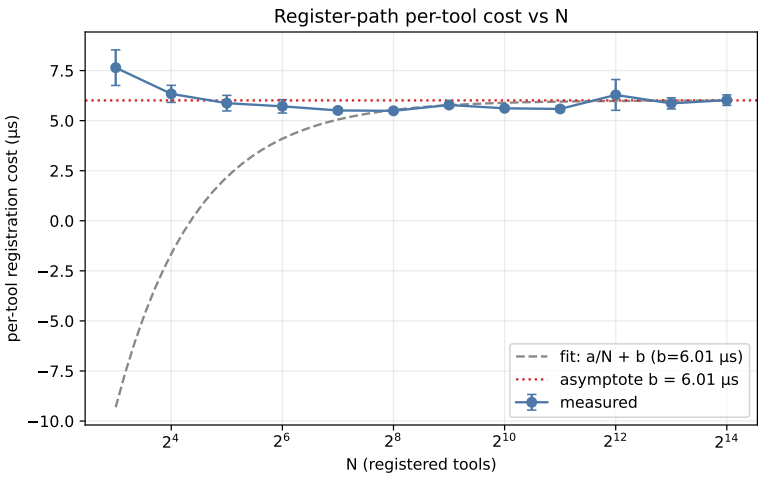


Figure 10: Registry register-path per-tool cost as a function of registry size N. The dashed line is the asymptotic per-tool cost $b = 6.011$ μs from the linear fit total_ms$(N) = a + bN$ ($R^2 = 0.9996$); the curve approaches this asymptote monotonically with no degradation at large N.

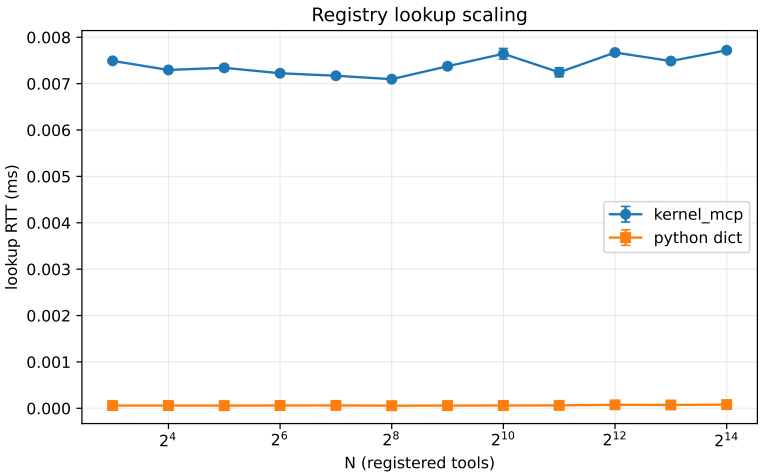


Figure 11: Steady-state `tool_id` lookup latency as a function of registry size N, on a log-N axis. The fitted log-linear slope is 2.9×10^{-5} ms per doubling (bootstrap 95% CI $[-2 \times 10^{-6}, +6.1 \times 10^{-5}]$, containing zero), consistent with the $O(1)$ xarray lookup invariant.

4.4 Security Evaluation

The latency and throughput numbers establish that kernel placement is not prohibitively expensive, but the argument for making the move hinges on whether it actually changes what attackers can do. The security evaluation targets four categories from the threat model in Chapter 2: identity spoofing (forging session identity to impersonate a registered agent), replay (reusing a stale approval ticket), substitution (tampering with `tool_id` or `tool_hash` to redirect execution), and escalation (bypassing the approval gate on a high-risk invocation).

Each system received 110 attack attempts in total (11 distinct cases $\times$ 10 repeats: 3 spoof, 4 replay, 3 substitution, 1 escalation). Outcomes are summarised in Table 9 and Figure 12.

Table 9: Category-level attack outcomes by system (110 attempts each: 3 spoof, 4 replay, 3 substitution, 1 escalation, × 10 repeats). The bypass rate measures the fraction of attempts that succeeded in circumventing the intended control. The user-space bypass rates reflect deliberate per-category instrumented weakening of the mediator and do not represent the performance of a fully hardened user-space implementation.

System	Bypasses	Total	Bypass rate
`userspace`	100	110	90.9%
`seccomp`	40	110	36.4%
`kernel`	0	110	0.0%

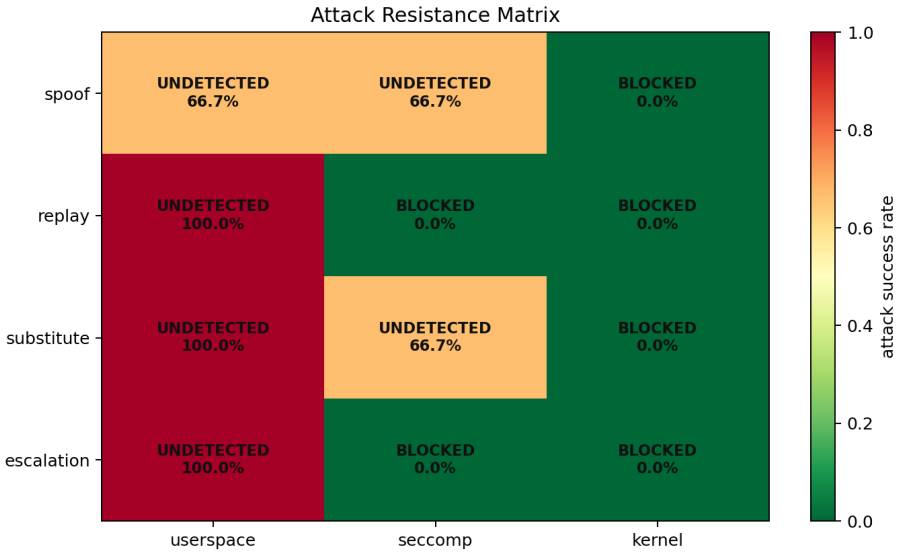


Figure 12: Attack bypass rate by category and system. Each cell shows the absolute count (successes/attempts) and the percentage bypass rate; darker cells indicate a higher bypass rate. The `kernel` column is uniformly zero across all four categories, while the two user-space baselines permit bypasses on the specific checks that their instrumented tamper profiles relax.

The plain user-space baseline fails to block 100 of 110 attempts (90.9% bypass). A caveat: the harness deliberately weakens the baseline's code paths per category (`tamper_session`, `tamper_approval`, `tamper_metadata`, `compromised_userspace`) so that each test exercises the specific check it targets. The 90.9% figure shows what a user-space mediator cannot structurally protect when its own state is under attack, not what a fully hardened version could achieve. The seccomp variant brings the bypass rate down to 36.4% through stricter checks and a narrower syscall surface, but admission is still a user-space decision. Attacks that do not need to cross a privilege boundary (particularly substitution and spoofing) still get through.

The kernel path blocks every one of the 110 attempts.

Spoofing fails because agent identity is tied to a kernel-registered identifier that the control

plane verifies independently of the daemon's session state. Replay fails because approval tickets are kernel-resident lifecycle objects with explicit `consumed` and `expired` fields; once a ticket has been advanced or has passed its validity window, no subsequent `APPROVAL_DECIDE` can reopen it. Substitution is blocked by the semantic hash check: regardless of whether an attacker modifies the `tool_hash` field in the request or replaces the tool endpoint at runtime, the kernel compares the submitted hash against the one recorded at registration and rejects any mismatch. A supplementary injection experiment confirms this independently: in 30 live `llm-app` → `mcpd` → `kernel` execution chains, every runtime hash substitution produced a `DENY` with `reason=hash_mismatch` at the kernel layer, regardless of which tool the LLM planner had selected.

One implementation detail of the ticket lifecycle strengthens the replay defence beyond what the category table directly measures: a ticket is bound not only to the session, agent, tool, and hash, but also to the exact `req_id` of the issuing request. A replay consume attempt that reuses a different `req_id` returns `approval_ticket_scope_mismatch`, making it substantially harder to exploit a stolen ticket identifier in a fresh request context.

Three caveats bound the interpretation of Table 9. First, each attack category maps one-to-one onto an enforcement mechanism in the kernel module. A 0% bypass rate on the kernel path is therefore a functional-correctness test of four specified invariants against inputs constructed to exercise exactly those invariants, not evidence of adversarial robustness against unknown attacks.

Second, the combined attack set is scoped to the stated control-plane invariants and does not include: attacks assuming compromise of the kernel module itself; attacks exploiting semantic vulnerabilities in tool logic; attacks requiring already-obtained privileged capabilities (such as `CAP_NET_ADMIN`); or attacks against the LLM prompt-construction pipeline. The 0% bypass rate is a statement about the specified control-plane invariants, not about the full attack surface of an LLM-integrated system.

Third, the user-space bypass rates reflect the instrumented tamper modes described above, not the performance of an optimally hardened user-space implementation.

4.5 Extended Threat Model

The 110-attempt category-level evaluation in Table 9 covers design-time invariants that the kernel module was built to enforce. A separate experiment targets three threat classes that sit outside that set and that a design-time review would not surface: a time-of-check-time-of-use (TOCTOU) race on the approval window, a cross-UID session hijack under adversarial Generic Netlink credentials, and dumb fuzzing of the `KERNEL_MCP` Generic Netlink family.

TOCTOU on the approval window. Two concurrent threads race against a high-risk tool: one repeatedly consumes approval tickets while the other re-registers the underlying tool binding, creating a window in which a ticket issued against one binding could in principle be consumed against another. Across 10 000 iterations, zero `ALLOW` outcomes were produced: 9985 requests were rejected with `hash_mismatch` at the kernel boundary, and the remaining 15 were deferred with `approval_ticket_unknown` because the consume attempt raced ahead of ticket issuance. The Clopper–Pearson 95% CI on the blocked rate is $[0.9996, 1.0000]$. The absence of breaches is consistent with the lock ordering in `kernel_mcp_cmd_tool_request`: the ticket consume and the tool-hash recomputation both acquire `tool_lock`, so a replacement that swaps the underlying binding cannot land between the two checks.

Cross-UID session hijack. A session is opened by UID A, and a second process running as UID B attempts three replay variants: blind replay, replay with a guessed session identifier, and replay with a leaked session identifier. Across 500 attempts, this experiment surfaced a defect that a design-time review of the kernel module had missed. In the version merged from the main evaluation branch, `kernel_mcp_cmd_tool_request` never compares `NETLINK_CB(skb)` peer credentials against the `uid` recorded at agent registration, so a crafted-credential replay from UID B reaches the request handler as if it had been sent by UID A. Of the 500 attempts, 326 were still blocked by downstream checks (mostly hash or binding mismatch against the replayed request body), but 174 passed every kernel-side check that the stock binary enforces and would have been admitted to tool execution. Those 174 are what the zero-bypass numbers in Table 9 do not cover, because the category-level harness never probes a second UID.

The fix is a six-line patch that walks `sock_i_uid(NETLINK_CB(skb).sk)` on entry to the handler and rejects the request with `-EPERM` on mismatch, gated behind a new `require_peer_cred` module parameter so that the A/B comparison can be run against a single binary. With the gate enabled, all 500 attempts are blocked at peer-credential verification before any registry work runs; the Clopper–Pearson 95% CIs for the two conditions do not overlap. The patch adds no measurable latency because the peer-credential read is on the same cache line as the existing `NETLINK_CB` access that the handler already performs. This is reported as an evaluation-discovered defect rather than buried: it is exactly the kind of invariant that a kernel-enforced control plane is supposed to hold, and the fact that the stock binary did not hold it until E4 ran is the strongest argument in the chapter for running the experiment at all.

Generic Netlink dumb fuzzer. The `KERNEL_MCP` family was exercised with a coverage-free structured fuzzer that iterates every defined `KMCP_CMD_*` opcode and applies per-`nlattr` bit flipping, length mutations, and type substitutions. Over a 30-minute fuzzing window, 880 640 mutated messages were delivered to the kernel module. Monitoring the kernel log buffer across the run (`dmesg` delta plus `kmemleak` scan) produced zero `oops`, `WARN`, `BUG`, `GPF`, or memory-leak

reports. Every malformed input was rejected with a deterministic `errno` drawn from a small set (`-EINVAL`, `-EBADMSG`, `-EPERM`). Figure 13 shows the distribution of returned errnos across the mutated message population. The fuzzer's design time was limited, so the result is read as a robustness smoke test rather than as evidence of adversarial hardness: it says that the Generic Netlink dispatch layer does not crash on malformed input within the tested 30-minute budget, not that the kernel module is free of latent memory-safety bugs under an unbounded adversary.

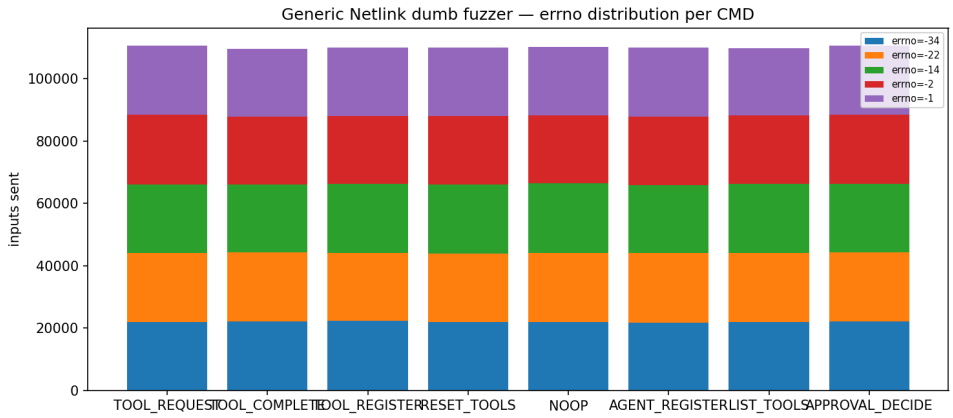


Figure 13: Errno distribution for the Generic Netlink dumb fuzzer, binned by the `KMCP_CMD_*` opcode under test. Every rejection falls into one of three deterministic errno classes (`-EINVAL`, `-EBADMSG`, `-EPERM`); zero mutated messages produced a kernel oops, `WARN`, or `kmemleak` report across 880 640 total inputs.

4.6 Daemon Failure and Recovery

A core architectural argument for kernel-held state is that it should outlive the process that registered it. To test this property, the daemon was forcibly interrupted mid-operation and control-plane state was observed before and after restart.

In both user-space systems, daemon failure is control-plane failure. Registered tool state, in-flight sessions, and pending approval tickets are all bound to the process that held them. When it dies, they are gone. Recovery means starting over or reloading from application-level logs, with no consistency guarantee.

The kernel path is different. Approval tickets, the tool registry, and agent registrations survive the crash intact and remain inspectable via `sysfs` without any daemon running. Session state is not preserved, since the binding context for active sessions is owned by the daemon process and is lost with it, but this is expected as session state is inherently tied to an active connection. The durability of deferred approvals and registry state allows `mcpd` to reconcile its manifest-derived view against the kernel-held view on restart and re-establish a consistent catalogue without issuing duplicate registrations.

The reconciliation strategy on restart is conservative (described in Section 3.9): cold start clears

and fully rebuilds the tool registry from manifests rather than attempting to reuse residual kernel state, even when the manifest set has not changed. Incremental reconciliation is already used while the daemon is running (only tools whose semantic or binding fingerprint changed are re-registered); extending that behaviour to the cold-start path, gated on a divergence signal, would reduce restart overhead and is a natural direction for future work.

4.7 Discussion

The security and performance results are easier to read as a pair than in isolation.

Consider the seccomp baseline. It does strictly more work than the plain user-space mediator: tighter syscall filters, extra input validation, audit logging. It still admits 36.4% of the attack attempts, and under a megabyte-class payload it is the slowest system in the table. More filtering around a mutable user-space process does not move the trust decision. The kernel path reaches a zero bypass rate not by filtering harder but because the admission state lives in a place no unprivileged process can reach. That is a structural property, and no amount of user-space hardening substitutes for it.

The 1 MB latency inversion is the same architectural fact showing up on the performance side, and it was not something we anticipated going in. BPF filtering pays a per-syscall cost that grows with the amount of socket traffic a request moves; kernel arbitration does not scale with payload size at all, because the arbitration decision is taken on a tiny fixed-size Generic Netlink message regardless of how big the downstream tool payload happens to be. By 1 MB this flips the expected ordering: the stronger enforcement boundary is also the faster one. The ablation in Table 7 says the same thing from a different angle. Of the 4.334 μs that kernel arbitration adds over the NOOP floor, the registry+lookup stage costs 0.109 μs and the rest is reply-skb construction; nothing in the fast path scales with the number of tools or with the request body. The registry scaling experiment extends this from microbench to catalogue size, showing that the lookup latency stays within a 0.6 μs band between $N = 8$ and $N = 16\,384$ tools and that registration itself runs at roughly 166 000 tools per second. None of these numbers are large enough to matter next to a single tool execution, which is the point.

The durability result is less dramatic to present, but it is arguably the most practically important of the set. When the daemon crashes, a purely user-space design loses everything it was holding: pending tickets, session context, registry state. Recovery means starting over. On the kernel path, a daemon restart is just a reconciliation pass against state that was never in the daemon to begin with. This is not a clever optimisation; it falls out of where the state lives. A forced restart mid-workflow is also, in practice, a far more common event than any of the controlled attack attempts the security evaluation tests.

The extended-attack results exposed a defect in the stock binary that the design-time harness did not cover. In the cross-UID experiment, `kernel_mcp_cmd_tool_request` did not compare

`NETLINK_CB(skb)` peer credentials against the UID recorded at agent registration, so requests from UID *B* reached the handler as if they had originated from UID *A*. Of the 500 attempts, 174 passed the checks enforced by the stock binary and would have been admitted to tool execution. We fixed this with a six-line peer-credential check on entry to the handler, guarded by a `require_peer_cred` module parameter so the A/B comparison could use the same binary. With the check enabled, all 500 attempts were blocked before any registry work ran, and the added comparison had no measurable latency cost. The practical takeaway is that peer-credential checks belong at the entry point of every privileged command handler, not as an afterthought.

What the evaluation does not settle is behaviour beyond the concurrency window it sweeps. Inside the clean window up to 100 concurrent requests the kernel path holds a stable $\sim$10% throughput gap against plain user-space without diverging; outside it, the global registry lock is an obvious suspect for a bottleneck and the current prototype does not decompose it. Per-tool or per-agent lock decomposition is the natural direction, but it sits outside the scope of this evaluation.

4.8 Threats to Validity

The results should be interpreted within four scope limitations: all experiments ran in a single VMware guest on an aarch64 host, so latency figures are environment-specific and generalisation to bare-metal or different kernel configurations is not established; the evaluation uses demonstration tools with predictable execution times and does not characterise tail latency under sustained load; the attack study is a controlled functional-correctness test of the four explicitly identified invariants and does not constitute a robustness assessment against unknown attacks, prompt injection, or privileged adversaries; and kernel-level admission control addresses only one layer of an AI-agent system, leaving semantic tool vulnerabilities, unsafe policy choices, and compromise outside the stated threat model unaddressed.

Chapter 5: Conclusion and Further Work

5.1 Conclusion

The broader question behind this project is what a traditional Linux system needs in order to host AI agents well. This work takes the narrower angle of asking it through the MCP protocol: if an agent's interaction with the outside world flows through a standardised tool-invocation layer, which parts of that layer genuinely need operating-system support, and which are fine where they are?

For the control-plane side of MCP, the answer is a small set of invariants: semantic consistency between a registered tool and the one actually executed, identity binding between a request and the session that made it, approval validity across the defer-approve-execute lifecycle, and state durability across gateway failure. These four properties are structurally hard to hold in a user-space gateway, because each one has to survive the failure or compromise of the very process that enforces it. Everything else belongs in user space, where flexibility matters and mistakes are cheap to fix.

`linux-mcp` is one concrete shape this split can take. A small Linux kernel module holds the registries, arbitrates admission, and owns the approval-ticket lifecycle behind a `CAP_NET_ADMIN`-gated Netlink interface; a user-space daemon loads manifests, derives semantic hashes, binds sessions to UDS peer credentials, and forwards admitted requests to tool services. Once the daemon has populated the kernel's view of the world, the integrity of that view no longer depends on the daemon staying alive or correct, and the tickets and registry entries stay in kernel memory after a daemon crash.

In the tested prototype, the kernel path enforced all specified control-plane invariants across the targeted attack set while both user-space baselines permitted meaningful bypass rates; latency and throughput stayed close to the user-space baseline throughout. A separate extended-attack rerun (TOCTOU, cross-UID session hijack, 30-minute Generic Netlink dumb fuzzer) surfaced a missing peer-credential check in the stock kernel module and closed it with a six-line patch at no measurable latency cost. These results are a functional-correctness test of specified invariants inside a single VM, not a claim of adversarial robustness or production-scale performance. They are nevertheless enough to show the split is workable: the enforcement gains are real, the overhead is small, and neither the LLM client nor tool authors need to change anything beyond providing a JSON manifest.

The position argued here is narrower than AIOS-style proposals that rebuild OS infrastructure around LLM abstractions, and less trusting than user-space frameworks that accept gateway correctness as given. Whether AI agents eventually deserve a full operating system of their own is an open question and probably a long way off; in the meantime, a conventional Linux

system extended with a few carefully scoped agent-aware mechanisms can already host an MCP control plane that neither a user-space gateway nor existing Linux security primitives can match alone. `linux-mcp` is one such mechanism for one such concern, and finding the others is the longer-term project.

5.2 Reflection

The initial framing of this project was too broad. Work became substantially clearer once the problem was narrowed to a specific question: which invariants break when the gateway crashes or misbehaves, and are those the ones that benefit from kernel enforcement? That reframing aligned the threat model, the implementation, and the evaluation criteria.

The kernel interface went through several candidates before settling on Generic Netlink. eBPF was considered first, and state storage turned out not to be the obstacle; `bpf_timer` and pinned maps are adequate for approval-ticket lifecycle. The real obstacle is the hook-attachment model: eBPF LSM fires on kernel operations, not on parsed JSON-RPC messages, so an in-kernel enforcer would have to re-parse MCP framing from raw socket bytes under verifier bounded-loop constraints on every invocation. LSM hooks were also considered, but they fire at the syscall boundary while tool identity is an application-layer concept, and the layers do not align. Generic Netlink supports the full lifecycle state the design needs, at the cost of a loadable module where a simpler policy hook would have been lighter.

Working across the kernel-user-space boundary was technically demanding. Attribute packing bugs in Netlink produce no compiler errors; they silently corrupt fields, surfacing only as unexpected kernel decisions deep into a test run. The most useful mitigation was writing standalone per-command test scripts before integrating with `mcpd`.

The most persistent design tension was the temptation to move more logic into the kernel. Each additional check there is one the gateway cannot bypass, which is attractive, but manifest parsing, schema validation, and tool execution belong in user space where iteration is fast and debugging does not require a module reload. The narrow kernel role was not a limitation; it was the point.

More broadly, the scope of a security claim matters as much as the claim itself. Framing `linux-mcp` as protecting four specific control-plane properties against a local unprivileged adversary made the evaluation honest and the argument defensible. A system that claims to solve the full AI-agent security problem is harder to evaluate and trust than one that solves a well-defined sub-problem and says so.

5.3 Further Work

Several directions follow naturally from the prototype.

First, kernel arbitration is intentionally minimal: tool and agent identity, semantic-hash consistency, and approval-ticket validity, but no richer policy framework. Per-tool permissions, role-scoped access control, rate limits, and execution budgets are the obvious next step, subject to the narrow-kernel principle that the kernel verifies policy membership but never computes policy decisions.

Second, the approval mechanism can be developed into a fuller governance path. The present design supports defer-and-decide control for sensitive operations, but real deployments will likely need approval delegation, bounded approval scope, revocation, expiry policies, and richer audit trails linking each decision to a concrete execution context. Strengthening this path would make `linux-mcp` more suitable for accountability and compliance contexts.

Third, recovery behaviour can be refined. The prototype preserves kernel-held registry and approval state across daemon failure, but cold-start reconciliation is unconditional, and daemon-local state (sessions and the `req_id`-to-ticket map) is discarded rather than reconstructed, so clients must reopen a session before resuming. A more mature design would gate the cold-start sweep on an observable divergence signal, carry richer recovery metadata for interrupted requests, and let clients resume a still-valid approval ticket without reopening. This matters most in long-running agent workflows, where failure and restart are routine.

Fourth, the evaluation can be extended substantially. The current registry-scaling experiment already pushes the kernel catalogue to 16384 tools with flat lookup latency across three orders of magnitude of N, but several adjacent regimes remain uncharacterised: bare-metal evaluation to eliminate hypervisor timing noise, sustained concurrency well beyond the clean-sweep ceiling of 100 concurrent requests (the `SOMAXCONN` saturation that blocked the present overload attempt needs addressing in the harness first), heterogeneous tool workloads with variable or network-dependent execution times, and p_{99} latency under sustained concurrency (the p_{95} characterisation in Chapter 4 does not extend that far). Comparison against additional hardened user-space designs would help separate the benefits of kernel placement from those achievable through improved user-space engineering alone.

Finally, this work opens a broader research question: which other agent-oriented abstractions deserve operating-system support? `linux-mcp` shows that a small but consequential part of the MCP control plane benefits from OS-level support without requiring AI agents to become first-class OS citizens or a new OS substrate. Similar arguments may extend to execution budgets, provenance across tool chains, durable approval logs, or agent-scoped resource accounting. Exploring these directions would clarify whether agent-aware OS support is best added incrementally, as in the present design, or eventually justifies a dedicated substrate.

References

[1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:221003629. 2023.

[2] Liu X, Yu H, Zhang H, et al. AgentBench: Evaluating LLMs as agents. arXiv preprint arXiv:230803688. 2024.

[3] Schick T, Dwivedi-Yu J, et al. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:230204761. 2023.

[4] Park JS, O'Brien J, Cai CJ, Morris MR, Liang P, Bernstein MS. Generative Agents: Interactive Simulacra of Human Behavior. In: UIST; 2023. .

[5] Qin Y, Liang S, Ye Y, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv preprint arXiv:230716789. 2024.

[6] Mei K, et al. AIOS: LLM Agent Operating System. arXiv preprint arXiv:240316971. 2024.

[7] Liu R, Zhe T, Wang D, et al. AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem. arXiv preprint arXiv:260308938. 2026.

[8] Model Context Protocol. Architecture Specification; 2025. `https://modelcontextprotocol.io/specification/2025-06-18/architecture`.

[9] Packer C, Fang V, Patil SG, Lin K, Wooders S, Gonzalez JE. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:231008560. 2023.

[10] Steinberger P. OpenClaw: Open-source Autonomous AI Agent; 2026. `https://openclaw.ai`.

[11] Nous Research. Hermes Agent; 2026. GitHub repository, accessed April 13, 2026. `https://github.com/nousresearch/hermes-agent`.

[12] Peter S, Li J, Zhang I, et al. Arrakis: The operating system is the control plane. In: OSDI; 2014. .

[13] Greshake K, et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with Indirect Prompt Injection. arXiv preprint arXiv:230212173. 2023.

[14] Tanenbaum AS, Bos H. Modern Operating Systems. 5th ed. Pearson; 2022.

[15] Chase H. LangChain; 2022. Open-source LLM application framework. `https://github.com/langchain-ai/langchain`.

[16] LangChain AI. LangGraph: A Framework for Building Stateful Multi-Actor LLM Applications; 2024. Open-source agent orchestration framework. https://github.com/langchain-ai/langgraph.

[17] Gravitas S. AutoGPT: An Autonomous GPT-4 Experiment; 2023. Open-source autonomous agent framework. https://github.com/Significant-Gravitas/AutoGPT.

[18] CrewAI Inc . CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents; 2024. Open-source multi-agent collaboration platform. https://crewai.com.

[19] Microsoft Corporation. Semantic Kernel: Open-Source SDK for LLM Integration; 2023. Released publicly March 2023; reached v1.0 in early 2024. https://github.com/microsoft/semantic-kernel.

[20] Wu Z, et al. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. arXiv preprint arXiv:240207456. 2024.

[21] HKUDS. OpenHarness: Open Agent Harness with a Built-in Personal Agent—Ohmo!; 2026. Initial open-source release v0.1.0: April 1, 2026. https://github.com/HKUDS/OpenHarness.

[22] Harness Inc . Harness AI DevOps Agent; 2025. AI-powered autonomous workers embedded in governed CI/CD pipelines. https://developer.harness.io/docs/platform/harness-aida/ai-devops/.

[23] Apple Inc . Private Cloud Compute: A new frontier for AI privacy in the cloud; 2024. https://security.apple.com/blog/private-cloud-compute/.

[24] MCP Core Maintainers. One Year of MCP: November 2025 Spec Release; 2025. Model Context Protocol Blog, accessed April 13, 2026. https://blog.modelcontextprotocol.io/posts/2025-11-25-first-mcp-anniversary/.

[25] Patil SG, Zhang T, Wang X, Gonzalez JE. Gorilla: Large Language Model Connected with Massive APIs; 2023.

[26] Liu Y, et al. Prompt Injection Attack against LLM-integrated Applications. arXiv preprint arXiv:230605499. 2023.

[27] Wang Z, Tu H, Zhang L, et al. Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw. arXiv preprint arXiv:260404759. 2026.

[28] Wang Y, Gao H, et al. A Systematic Security Evaluation of OpenClaw and Its Variants. arXiv preprint arXiv:260403131. 2026.

[29] Wang G, Xie Y, Jiang Y, Mandlekar A, Xiao C, Zhu Y, et al.. Voyager: An Open-Ended Embodied Agent with Large Language Models; 2023.

[30] Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, et al.. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering; 2024.

[31] Lim SY, Agrawal S, Han X, Eyers D, O'Keeffe D, Pasquier T. Securing Monolithic Kernels using Compartmentalization. arXiv preprint arXiv:240408716. 2024.

[32] Schneider M, Masti RJ, Shinde S, Capkun S, Perez R. SoK: Hardware-supported Trusted Execution Environments. arXiv preprint arXiv:220512742. 2022.

[33] Lefeuvre H, Dautenhahn N, Chisnall D, Olivier P. SoK: Software Compartmentalization. arXiv preprint arXiv:241008434. 2024.

[34] Ji Z, Wu D, Jiang W, Ma P, Li Z, Gao Y, et al. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework. arXiv preprint arXiv:260111893. 2026.

[35] Shi T, He J, Wang Z, Wu L, Li H, Guo W, et al. Progent: Programmable Privilege Control for LLM Agents. arXiv preprint arXiv:250411703. 2025.

[36] Linux Kernel Documentation. Seccomp BPF; 2026. `https://docs.kernel.org/userspace-api/seccomp_filter.html`.

[37] Linux Kernel Documentation. Linux Security Modules (LSM); 2026. `https://docs.kernel.org/admin-guide/LSM/index.html`.

[38] Linux Kernel Documentation. AppArmor; 2026. `https://docs.kernel.org/admin-guide/LSM/apparmor.html`.

[39] Linux Kernel Documentation. SELinux; 2026. `https://docs.kernel.org/admin-guide/LSM/SELinux.html`.

[40] Linux Kernel Documentation. Landlock; 2026. `https://docs.kernel.org/userspace-api/landlock.html`.

[41] Wright C, Cowan C, Smalley S, Morris J, Kroah-Hartman G. Linux Security Modules: General Security Support for the Linux Kernel. In: Proceedings of the 11th USENIX Security Symposium. USENIX Association; 2002. p. 17-31.

[42] Salaün M. Landlock LSM: Toward Unprivileged Sandboxing. In: Linux Security Summit; 2017. .

[43] Watson RNM, Anderson J, Laurie B, Kennaway K. Capsicum: Practical Capabilities for UNIX. In: Proceedings of the USENIX Security Symposium; 2010. .

[44] Brimhall B, Garrard J, de la Garza C, Coffman J. A Comparative Analysis of Linux Mandatory Access Control Policy Enforcement Mechanisms. In: Proceedings of the 16th European Workshop on System Security (EuroSec); 2023. .

[45] Stevens WR, Rago SA. Advanced Programming in the UNIX Environment. 3rd ed. Addison-Wesley; 2013.

[46] Bishop M. Computer Security: Art and Science. Addison-Wesley; 2003.

[47] Lee Y, Min C, Lee B. ExpRace: Exploiting Kernel Races through Raising Interrupts. In: Proceedings of the 30th USENIX Security Symposium. Vancouver, BC; 2021. p. 2363-80.

[48] Klein G, Andronick J, Elphinstone K, Heiser G, Cock D, Derrin P, et al. seL4: Formal Verification of an Operating-System Kernel. Communications of the ACM. 2010;53(6):107-15.

[49] Linux Kernel Documentation. Netlink Socket Family; 2026. `https://docs.kernel.org/userspace-api/netlink/intro.html`.

[50] West RM. Best practice in statistics: Use the Welch t-test when testing the difference between two groups. Annals of Clinical Biochemistry. 2021;58(4):267-9.

Acknowledgment

I would like to express my heartfelt gratitude to everyone I have met over the past four years at university. Their kindness, encouragement, and quiet influence have shaped not only this project, but also the person I have become.

Appendices

Disclaimer

This report is submitted as part requirement for the undergraduate degree programme at Queen Mary University of London, and Beijing University of Posts and Telecommunications. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

BUPT No.: 2022213192

QMUL No.: 221166471

Full Name (Pin Yin): Xiheng LI

Full Name (Chinese): 李溪恒

Signature:

Date:27-04-2026

Acknowledgement of using Generative AI

I acknowledge the use of the following Generative AI tools:

Publisher	**Name, version, URL**	**Use**	**Section**
OpenAI	Codex (GPT-5-based coding assistant) `https://openai.com/` ChatGPT (GPT-5.4) `https://chatgpt.com/`	Used for translating support during report preparation.	Chapter 2; Chapter 4; Part of the source code
Anthropic	Claude Code (claude-sonnet-4-6) `https://claude.ai/code`	Used for grammar editing, and academic writing support during report preparation. Also, part of the scripts and source code are done by vibe-coding using Claude Code.	Chapter 2; Chapter 4; Part of the source code

I confirm the submitted work is my own work.

Project specification

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 1 - Supervisor

论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol		
题目分类 Scope	Research	Software Development	Software
主要内容 Project description	As AI agents increasingly function as autonomous computing entities capable of learning, reasoning, and collaboration, the fundamental limitations of traditional operating systems (OS) have become apparent, creating a need for a new generation of agent-oriented OS. This thesis focuses on developing a prototype of an agent-native OS centered around the Model Context Protocol (MCP). Specifically, it proposes an MCP-enabled kernel module that provides unified interfaces for AI agents to register, exchange contextual information, and coordinate computational tasks. Furthermore, it investigates how MCP-based kernel-level scheduling mechanisms can manage computational resources among multiple agents to ensure fairness, efficiency, and security. The results aim to lay the groundwork for future operating systems in which AI agents act as first-class citizens within the computing environment.		
关键词 Keywords	AI Agent, MCP, kernel		
主要任务 Main tasks	1 Analyze the evolution of OS architectures from process-centric to agent-centric paradigms and identify the key design principles of agent-native OS.		
	2 Study the internal structure of the MCP protocol and design how to provide its functionalities from the kernel level.		
	3 Implement a MCP-enabled kernel module and design the exposed APIs.		
	4 Evaluate the performance and scalability of the MCP-enabled kernel model, and discuss how it achieves efficiency, fairness, and security.		
主要成果 Measurable outcomes	1 A comprehensive review chapter comparing traditional OS abstractions with emerging requirements on OS from AI agents.		
	2 A detailed design discusses how MCP interfaces can be mapped to system calls or other interfaces the kernel exposes to user-level applications, and how MCP functionalities are implemented in the kernel.		
	3 Implement a working prototype of the MCP-enable kernel module, along with API documentation describing the exposed interfaces.		

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 2 - Student

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				
论文概述 Project outline **Write about 500-800 words** **Please refer to Project Student Handbook section 3.2**	This project aims to explore the design of an AI-Native Operating System (AI-Native OS) based on the Model Context Protocol (MCP). The system integrates a kernel-level MCP module, enabling native agent registration, context exchange, and scheduling. Through the integration of these layers, the project seeks to build a secure, efficient, and extensible foundation for next-generation operating systems where AI agents are treated as first-class computing entities. This project bridges MCP-based agent systems and next-generation operating system design, integrating secure automation with kernel-level scheduling. It aims to establish a secure and scalable foundation for future AI-native computing environments. The project addresses the needs of OS researchers and AI developers who require: 1. Unified interfaces for agent registration and context sharing; 2. Extensible OS infrastructure for managing heterogeneous agents. The developer feedback regarding system usability, safety, and performance. The project adopts a bottom-up system design combining kernel engineering and AI-driven automation. Key methodologies include: 1. Implementation of agents registration, context sharing, and cooperative scheduling. 2. Scheduling algorithm to balance fairness and resource efficiency using contextual metadata. The experiments will test the following hypotheses: 1. MCP-based kernel scheduling enhances fairness and efficiency compared with traditional schedulers. 2. The integrated kernel–server architecture provides lower latency and higher scalability than user-space MCP implementations. 3. Performance metrics include latency, CPU utilization, security violation rate, and scalability under multi-agent workloads. The experiments will be conducted in the following environment: 1. Programming Languages: TBD (kernel module), Python (testing and generation) 2. Software Tools: GCC, Clang, Docker, GDB, FastMCP, LangChain 3. Operating Environment: Linux Kernel 6.x (Ubuntu 22.04) 4. Hardware: x86-64 workstation or virtual machine				

	Background materials Zhang X, Zhang Y. Planet as a Brain: Towards Internet of AgentSites based on AIOS Server. arXiv preprint arXiv:2504.14411; 2025. Available from: https://arxiv.org/abs/2504.14411 Hu X, Xiong T, Yi B, et al. OS Agents: A Survey on MLLM-based Agents for General Computing Devices Use. arXiv preprint arXiv:2508.04482; 2025. Available from: https://arxiv.org/abs/2508.04482 Koubaa A. Agent Operating Systems (Agent-OS): A Blueprint Architecture for Real-Time, Secure and Scalable AI Agents. TechRxiv; 2025. Available from: https://www.techrxiv.org/users/689908/articles/1330306 Mastouri M, Ksontini E, Kessentini W. Making REST APIs Agent-Ready: From OpenAPI to MCP Servers for Tool-Augmented LLMs. arXiv [Preprint]. 2025;arXiv:2507.16044. Available from: https://arxiv.org/abs/2507.16044 Widjaja F, Chen Z, Zhou J. BioinfoMCP: A Unified Platform Enabling MCP Interfaces in Agentic Bioinformatics. arXiv [Preprint]. 2025;arXiv:2510.02139. Available from: https://arxiv.org/abs/2510.02139 Wang Z, Chang Q, Patel H, Biju S, Wu CE, Liu Q, et al. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. arXiv [Preprint]. 2025;arXiv:2508.20453. Available from: https://arxiv.org/abs/2508.20453 Tanenbaum AS, Bos H. Modern Operating Systems. 5th ed. Pearson; 2022. OpenAI. Model Context Protocol (MCP) Specification. 2024. Available from: https://github.com/modelcontextprotocol OpenAI Research. MCP Security Framework and Safe Context Exchange. arXiv preprint; 2025. Almeida R. On the Stability of Memory-Dependent Multi-Agent Systems. Future Generation Computer Systems. 2025 (In Press). Available from: https://www.sciencedirect.com/science/article/pii/S0016003224008238
道德规范 **Ethics**	Please confirm by checking the box: ☒ I confirm that I have discussed ethical issues with my supervisor.

Please discuss ethical issues with your supervisor. **Please refer to Project Student Handbook section 4.1 and Project Ethics Guidance**	Summary of ethical issues: This project does not involve any ethical issues.
中期目标 **Mid-term target.** **It must be tangible outcomes, E.g. software, hardware or simulation.** **It will be assessed at the mid-term oral.**	A working prototype demonstrating preliminary kernel-level MCP module. Deliverables include: 1. A thorough survey about the evolution from traditional process-centric OS to agent-centric AI-native OS 2. Kernel module implementing agent registration & context exchange;

Work Plan (Gantt Chart)

Fill in the sub-tasks and insert a letter X in the cells to show the extent of each task

	Nov 1-15	Nov 16-30	Dec 1-15	Dec 16-31	Jan 1-15	Jan 16-31	Feb 1-15	Feb 16-28	Mar 1-15	Mar 16-31	Apr 1-15	Apr 16-30
Task 1 Analyze the evolution of OS architectures from process-centric to agent-centric paradigms and identify the key design principles of agent-native OS.												
Review traditional OS architectures	X	X	X									
Analyse agent computing requirements, context management, and fairness	X	X	X									
Compare traditional OS abstractions vs AI-native demands			X	X	X							
Formulate design principles and theoretical framework for AI-Native OS			X	X	X							
Task 2 Study the internal structure of the MCP protocol and design how to provide its functionalities from the kernel level.												
Analyse structure, context flow, and core mechanisms of MCP.	X	X	X									
Map MCP functions to kernel system calls			X	X	X							
Define kernel-side data structures (context tables, registry).					X	X	X					
Write design specification and interface documentation.					X	X	X					
Task 3 Implement a MCP-enabled kernel module and design the exposed APIs.												
Implement a lightweight user-space daemon that loads the MCP manifest, manages tools.					X	X	X					
Build the tool adapter layer for external tools.					X	X	X					
Achieve working prototype and demo integration workflow.					X	X	X					
Build an automated test runner that compares MCP tool outputs with baseline executions to verify correctness.							X	X	X			
Task 4 Evaluate the performance and scalability of the MCP-enabled kernel model, and discuss how it achieves efficiency, fairness, and security.												
Define performance metrics and design experiments					X	X	X					
Conduct performance & scalability experiments under multi-agent workloads.							X	X	X	X		
Run static & runtime security tests							X	X	X	X		

Summarize results, visualize data, evaluate the result									X	X	X	X

Early-term progress report

北京邮电大学 本科毕业设计（论文）初期进度报告

Project Early-term Progress Report

学院 **School**	International School	专业 **Programme**	**Telecommunications Engineering with Management**		
姓 **Family name**	Li	名 **First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	班级 **Class**	2022215107
论文题目 **Project Title**	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

1. Introduction

The integration of Large Language Models (LLMs) into operating systems has catalyzed a paradigm shift from static instruction execution to dynamic, autonomous agent behavior. However, current frameworks operate atop general-purpose operating systems that lack native abstractions for managing the lifecycle of LLM inference. Relying solely on traditional process scheduling results in uncontrolled contention for the limited context window and lacks isolation mechanisms to prevent agents from monopolizing system resources or accessing unauthorized memory regions.

This project proposes an AI-Native Operating System architecture that integrates the Model Context Protocol (MCP) [1] directly into the kernel subsystem. By elevating "Agent Context" and "Tools" to first-class kernel citizens, we aim to replace passive user-space management with active kernel-level scheduling and security enforcement, thereby bridging the gap between traditional OS resource management and autonomous agent workflows.

2. Literature Review

2.1 Limitations of User-Space Agent Frameworks

Current agent architectures, such as AutoGPT [2] and Generative Agents [3], operate as user-space processes atop general-purpose operating systems. However, these traditional OS environments lack native abstractions for LLM-centric workflows. Qin et al. [4] pointed out that existing frameworks lack a unified mechanism to manage LLM resource contention. Instead of scheduled access, agents employ an inefficient "trial-and-error" approach: they

attempt to load prompts into GPU memory until hitting hardware limits (e.g., CUDA out-of-memory exceptions), forcing expensive deallocation and multiple retry attempts that severely degrade system throughput.

Furthermore, current general-purpose OS abstractions treat Agent Context as opaque data blobs, forcing frameworks to manage state retrieval sequentially in user-space, ignoring potential data-level parallelism. Lacking native parallel primitives for "Agent Context," the system fails to leverage multi-core parallelism for concurrent reasoning tasks, forcing agents to process information strictly linearly.

2.2 The "LLM as OS" Metaphor vs. Reality

Recent works like AIOS [5] , MemGPT[6] , and Xserve [7] have popularized the concept of an "OS for LLMs." MemGPT introduces a virtual memory hierarchy to manage limited context windows, Xserve introduces a system-level abstraction and service, which utilizes a virtual capability layer with a unified API to streamline the integration of AI into applications, while AIOS proposes an LLM-centric scheduler that isolates resources and services into an AIOS kernel to improve scheduling, context management, and resource utilization efficiency. However, a critical analysis reveals fundamental architectural limitations in these systems, the MemGPT focus mainly on context memory and text generation, which is not enough for an AI agent operating system. While AIOS demonstrates improved throughput across various agent benchmarks, it operates entirely as a user-space application (running on the Python interpreter). Consequently, it lacks strict isolation: its 'Access Manager' relies on high-level logic checks, meaning a malicious agent could potentially crash the host process or bypass restrictions. Furthermore, AIOS relies on software-level context interruption rather than true hardware preemptive scheduling of GPU resources, preventing it from enforcing hard real-time guarantees for mission-critical agent responses.

Also, in the security level, the existing AI operating systems lack isolation. A malicious agent can easily crash the host Python process or access unauthorized memory, as demonstrated in recent prompt-injection attacks on agentic systems [8][9].

Some mobile operating systems have begun integrating machine learning workloads directly into the system infrastructure beyond simple application-level and configured out some solutions to tackle the above problems.

In 2024, Apple introduced a system-wide intelligence layer deeply integrated into iOS and macOS[10]. A key component of its architecture is the combination of App Intents and the Semantic Index. App Intents is a Swift-based protocol that redefines application functionalities using a standardized intent schema and exposes them to the operating system, enabling the LLM to invoke applications to fulfill user instructions. The Semantic Index organizes user data across applications to provide context-aware capabilities. Apple adopts a hybrid execution model, processing sensitive tasks on device while offloading more computationally intensive tasks to Private Cloud Compute.

Google introduced Private Compute Core to bring a new layer of privacy to the Android ecosystem. for OS-level ML features [11]. Private Compute Core is an open source, secure environment that is isolated from the rest of the operating system and apps. It ensures that sensitive data processed by ML models stays isolated from the network via strict API boundaries, communicating with the cloud only through Private Compute Services for model updates [12].

While these industrial solutions validate the necessity of OS-level AI integration, they are proprietary and siloed ecosystems. They lack a unified, open interface for third-party agents to interact with the kernel's context management. This project differs by proposing an open-source, kernel-native implementation using the standardized MCP, aiming to democratize system-level agent capabilities for the Linux ecosystem.

2.3 Kernel-Level Opportunities and the MCP Protocol

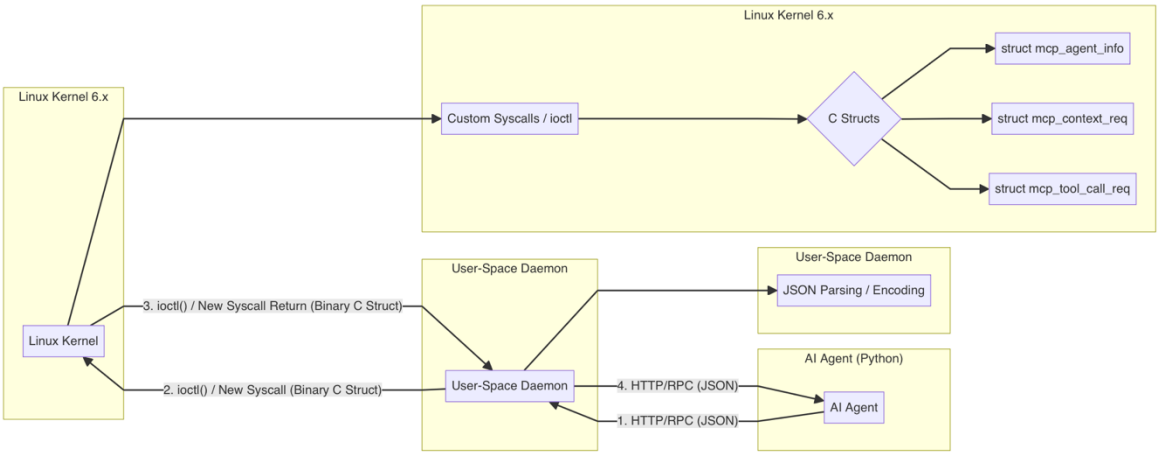
The MCP provides a standardized schema for linking agents to data, and it can effectively prevent the malicious instructions sent by agents. However, current MCP implementations are exclusively high-level (TypeScript/Python). There is a distinct lack of research on implementing MCP primitives as System Calls. This project fills this gap by exploring a Kernel-Level MCP Module to offer lower latency and mandatory access control.

3. Project Progress Summary

3.1 Completed Work

In accordance with Task 1 and the early stages of Task 2 in the project work plan, the following progress has been made:

- Theoretical Framework Analysis: I have completed a comparative analysis between traditional process-centric paradigms (Process Control Blocks - PCB) and agent-centric demands (Agent Context Blocks). I determined that the Linux task_struct needs to be extended or pointed to by a new auxiliary structure to hold MCP metadata.
- MCP Protocol Analysis: I have deeply analyzed the MCP specification and security, specifically the lifecycle of initialization and the prompts, resources, and tools primitives. I have drafted a mapping table that translates MCP JSON-RPC messages into theoretical C-language kernel structures.



- Environment Setup: A development environment has been successfully established on my MacBook using VMware, the Linux version is Ubuntu 22.04 LTS.
    - Compiled Linux Kernel 6.x source code to ensure the environment is ready for custom module insertion.
    - Installed and tested FastMCP to understand the standard user-space behavior of the MCP protocol.
    - Configured QEMU/KVM for safe kernel debugging (GDB) to prevent crashing the host machine during development.

In the future, I will deploy the system on a high-performance server to evaluate scalability and accurate performance metrics under multi-agent workloads

3.2 Comparison with Work Plan

The project is currently on schedule.

- Planned: Analyze OS architectures and MCP internal structure by mid-January.
- Actual: The literature review and architectural analysis are complete. The mapping of MCP functions to kernel system calls (Task 2) is currently in progress.
- Adjustment: The complexity of parsing JSON (used by MCP) inside the kernel was identified as a technical risk. I have decided to implement a lightweight binary serialization protocol for the kernel boundary, with a user-space daemon handling the JSON conversion, which slightly refines the approach in Task 3.

4. References

[1]. Model Context Protocol, "Model Context Protocol Specification," 2024. [Online]. Available: https://spec.modelcontextprotocol.io/

[2]. T. Richards, "Auto-GPT: An Autonomous GPT-4 Experiment," GitHub Repository, 2023. [Online]. Available: https://github.com/Significant-Gravitas/Auto-GPT

[3]. J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. ACM Symp. User Interface Softw. Technol. (UIST), 2023.

[4]. Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. ICLR, 2024.

[5]. Kai Mei et al., "AIOS: LLM Agent Operating System," arXiv preprint arXiv:2403.16971, 2024.

[6]. C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560, 2024.

[7]. Jinrong Yang, Zimeng Wang, Rong Chen, and Haibo Chen. 2025. A System-level Abstraction and Service for Flourishing AI-powered Applications. In Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25). Association

for Computing Machinery, New York, NY, USA, 106–114. https://doi.org/10.1145/3725783.3764406

[8]. OWASP GenAI Security Project, "LLM01: Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[9]. F. Liu et al., "Prompt Injection attack against LLM-integrated Applications," arXiv preprint arXiv:2306.05499, 2023.

[10]. Apple Inc., "Apple Intelligence: System Architecture and Privacy," Apple Platform Security Guide, 2024. [Online]. Available: https://security.apple.com/documentation/

[11]. Google. Android Private Compute Core: Technical Whitepaper. Google Open Source Security, 2022. [Online]. Available: https://services.google.com/fh/files/misc/android-security-paper-2024.pdf

[12]. Google. Private Compute Services. GitHub Repository, 2022. [Online]. Available: https://github.com/google/private-compute-services

是否符合进度？On schedule as per GANTT chart?
YES

下一步 Next steps:

Kernel Data Structure Definition: Define the struct mcp_agent and struct mcp_context in C code within the kernel module to store agent metadata.

Kernel Module Skeleton: Write the initial Character Device Driver (/dev/mcp_core) that allows user-space processes to "register" as agents via ioctl calls.

Prototype Integration: Develop a minimal user-space Python script using ctypes to interact with the kernel module, demonstrating basic agent registration and context retrieval.

Mid-term progress report

北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				
是否完成任务书中所定的中期目标？Targets met (as set in the Specification)? YES					

已完成工作 Finished work:

1. Survey Results

This study first conducts a comprehensive survey on the evolution of operating system architectures, systematically analyzing the paradigm shift from traditional process-centric models to agent-centric AI-native operating systems. Specifically, this survey identifies a structural transition across four core dimensions:

Shift in Computational Abstraction: The system's first-class citizen evolves from the "process" to the "agent," shifting the computing paradigm from deterministic execution to LLM-driven, intent-oriented generation.

Reconstruction of Resource Management: Bottlenecks transition from traditional memory allocation to LLM context windows, token consumption, and API quotas, requiring novel multi-agent scheduling algorithms.

Semanticization of IPC: Inter-module communication shifts from low-level byte streams to highly semantic interactions, utilizing JSON schemas and standardized protocols like the Model Context Protocol to route data between agents and tools.

Redefining Security Boundaries: To mitigate risks like logical privilege escalation caused by model hallucinations, system security must upgrade from hardware-based memory isolation to semantic validation and capability hash-binding.

Against this evolutionary backdrop—particularly the shift towards semantic IPC and the need for redefined security boundaries—this work further investigates the internal structure of MCP and examines how tool invocation is handled in current AIOS prototypes and multi-agent orchestration environments.

A preliminary technical analysis of these existing environments reveals three major limitations. First, tool execution is typically governed purely in user space, making it difficult to enforce strict admission control at the operating-system level. Second, while logging may exist, structured auditing and accounting are rarely maintained as kernel-backed registries with explicit counters and completion tracking. Third, fairness under multi-agent contention is rarely modeled explicitly; resource contention is usually handled heuristically with limited quantitative evaluation.

These findings directly motivated the core design principle of this project: governance must be strictly enforced by a kernel control plane, while semantic parsing and tool execution remain securely isolated in user space.

2. Requirement Analysis

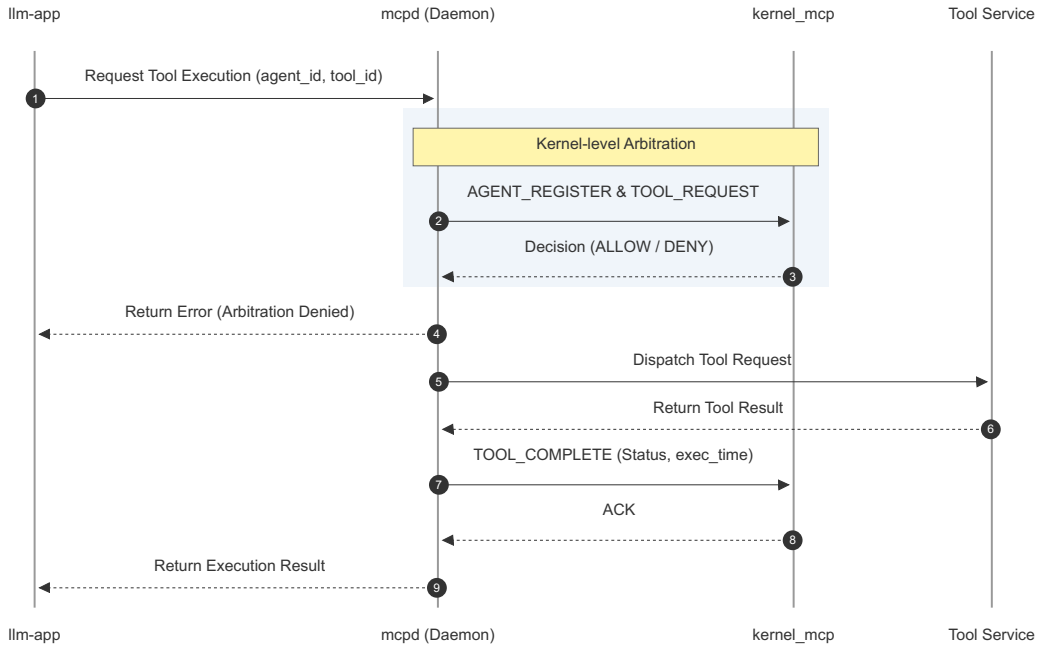
The system requirements are systematically categorized into functional requirements, architectural constraints, and non-functional requirements to ensure robust tool invocation and governance.

Functional Requirements: First, the system must provide explicit kernel-level arbitration. Every tool invocation request must yield a deterministic governance decision from the set {ALLOW, DENY, DEFER} before execution can proceed. Second, the system must support semantic discovery, enabling LLM-based clients to dynamically retrieve tool descriptions, input schemas, and execution examples for automated tool selection. Third, it must support cross-application tool invocation, requiring tools to be hosted and managed as independent resident services rather than ad-hoc inline scripts.

Architectural Constraints: To minimize kernel complexity and mitigate security risks, the system imposes a strict data-plane constraint: JSON parsing is strictly prohibited inside the kernel. The kernel boundary must only accept structured, fixed-size governance attributes (e.g., tool_id, agent_id, perm, cost, and tool_hash).

Non-Functional Requirements: From a non-functional perspective, the system must enforce strict semantic consistency between user-space application manifests and kernel-side registries to prevent semantic drift or substitution attacks. Additionally, the system must guarantee measurable fairness under concurrent multi-agent workloads, ensuring that resource contention is governed quantitatively rather than heuristically. Finally, the system must provide high runtime observability, exposing governance states, audit counters, and registry data to facilitate monitoring and debugging.

3. System Design



The implemented architecture is organized into four layers: an llm-app client layer, an mcpd gateway layer, a kernel_mcp governance layer, and a resident tool service execution layer. At runtime, the llm-app sends a framed JSON request to mcpd through a centralized Unix Domain Socket. The gateway performs routing using semantic metadata and issues a structured TOOL_REQUEST to the kernel via Generic Netlink, carrying only fixed-size governance attributes (agent_id, tool_id, perm/cost references, tool_hash, and related control fields). The kernel returns an arbitration decision (ALLOW, DENY, or DEFER). When ALLOW is returned, mcpd forwards the full JSON payload to the corresponding application-specific resident service endpoint. After

execution, mcpd reports completion back to the kernel via TOOL_COMPLETE, including status and latency, and returns the result to the client.

This design enforces a strict control-plane/data-plane separation. The kernel implements the control plane and does not parse or buffer unbounded JSON payloads. The data plane is handled entirely in user space: mcpd is the sole gateway that dispatches requests to resident tool services. Under the current threat model, clients do not directly interact with tool-service IPC endpoints; tool services accept requests routed through mcpd, which reduces the risk of bypassing kernel arbitration.

To prevent semantic drift between user-space manifests and kernel-side governance records, the system uses a canonical hash-binding mechanism. The hash is computed over stable capability attributes (tool_id, name, perm, cost, description, input_schema, and examples) while excluding runtime metadata such as endpoint paths and process identifiers. During startup, reconcile_kernel.py performs one-to-one comparison between manifest-defined tool entries and kernel registry entries; any mismatch triggers fail-fast termination. During runtime arbitration, tool_hash consistency checks provide an additional guardrail against executing a tool definition whose semantics diverge from the kernel-approved baseline.

4. Prototype Implementation

While the early-term report proposed using a character device and ioctl for kernel-user communication, the implementation transitioned to Generic Netlink. This adjustment provides a more structured and extensible IPC mechanism for multiplexing tool requests compared to legacy ioctl boundaries. A functional prototype has been implemented across kernel space and user space.

On the kernel side, the kernel_mcp module provides Generic Netlink commands for tool/agent registration, request arbitration, and completion reporting. It maintains in-kernel registries for tools and agents, implements per-agent token buckets for rate governance, and updates audit/accounting counters. These counters and registry snapshots are exposed via sysfs for observability. Basic stability checks were conducted through repeated module load/unload cycles, and no kernel warnings or crashes were observed during these tests.

On the user-space side, the mcpd gateway loads application manifests, computes canonical semantic hashes, performs startup reconciliation, enforces arbitration outcomes, forwards requests to resident tool services, and reports completion back to the kernel. For client-facing discovery, mcpd exposes semantic-only tool fields (descriptions, schemas, and examples) while avoiding leakage of runtime routing details such as service endpoints.

In the execution layer, multiple tools were migrated to persistent resident services over Unix Domain Sockets. A shared service library implements a 4-byte big-endian length-prefixed JSON protocol and supports concurrent handling via a thread-per-connection model. Each tool module supports both a debug mode and a resident service mode for development convenience. Security-oriented validations were added where applicable, including path restriction checks for file-related tools, safe evaluation constraints for expression-based tools, and basic schema/parameter validation.

These manifest snippets below illustrate how linux-mcp separates semantics from governance. The fields description, input_schema, and examples define tool semantics for discovery and selection in the llm-app layer, while runtime routing metadata (mode, endpoint, service_path, app_impl) is consumed by mcpd to dispatch requests to the correct resident tool service. Governance-related attributes form the canonical binding used for kernel-side arbitration and for startup reconciliation, ensuring a strict one-to-one correspondence between user-space manifests and the kernel registry. This design keeps JSON payload parsing and natural-language semantics in user space, while allowing the kernel control plane to enforce admission control and accounting using fixed-size structured identifiers.

```
{
  "app_id": "file_manager_app",
  "app_name": "File Manager App",
```

```
  "mode": "uds_service",
  "endpoint": "/tmp/linux-mcp-apps/file_manager_app.sock",
  "service_path": "tool-app/app_service.py",
  "app_impl": "tool-app/apps/file_manager_app.py",
  "tools": [
    {
      "tool_id": 12,
      "name": "file_delete",
      "perm": 1,
      "cost": 1,
      "handler": "file_delete",
      "description": "Delete a file (or directory with recursive=true) under repository root with safety
checks.",
      "input_schema": {
        "type": "object",
        "required": ["path"],
        "properties": {
            "path": { "type": "string" },
            "recursive": { "type": "boolean" },
            "allow_missing": { "type": "boolean" }
            },
        "additionalProperties": false
      },
      "examples": [
        { "path": "tmp/demo_created_by_mcp.txt" },
        { "payload": { "path": "tmp/old_logs", "recursive": true }}
      ]
    }
  ],
…
}
```


5. Functional Validation and System Integration

At the current stage, experimental validation focused on end-to-end system integration and functional correctness. Build validation confirmed the successful compilation of all kernel modules and C components, as well as the stable execution of the Python-based gateway and tool services. For LLM integration, the llm-app client was successfully connected to the DeepSeek large language model via API. Tests demonstrated that DeepSeek could correctly comprehend the provided tool manifests and generate well-formed JSON tool-invocation requests according to the protocol.

Crucially, the end-to-end closed-loop workflow was successfully verified across multiple different tools. The runtime observations confirmed the following sequence: user inputs trigger LLM responses; the llm-app submits tool requests to the mcpd gateway; the gateway initiates a governance check at the kernel level; the kernel successfully performs arbitration to evaluate the request; upon an ALLOW decision, the gateway dispatches the execution to the correct external tool-app; the tool executes its internal logic properly; and finally, the result is accurately propagated back to the llm-app. This functional testing demonstrates that the framework achieves its primary goal: enabling LLM-driven tool execution while enforcing strict kernel-level scrutiny.

6. Observability and GUI Integration

A graphical demo client was implemented in the llm-app layer to support interactive end-to-end validation. The GUI is built with PySide6 and connects to the gateway exclusively through the

centralized Unix Domain Socket interface exposed by mcpd. It provides an interactive workflow that mirrors the CLI pipeline: the GUI refreshes the application catalog via {"sys":"list_apps"}, retrieves tool catalogs via {"sys":"list_tools"}, and issues execution requests using {"kind":"tool:exec", ...} so that every invocation still goes through kernel arbitration and completion reporting enforced by mcpd .

The GUI exposes two practical observability surfaces during demonstration. First, it visualizes the discovered app/tool catalog after refreshing from mcpd, making it easy to confirm that the client only sees semantic fields and the tool hash, rather than runtime routing details . Second, for each user input, it shows the selected app and tool, the selector source and rationale, the request id, and the execution response including timing (t_ms), which helps trace the full loop in a single screen . The GUI reuses the same selection logic shared with the CLI, allowing consistent behavior across interactive and scripted experiments .

7. Summary

A functional linux-mcp prototype has been successfully implemented and validated, achieving kernel-governed tool invocation while strictly isolating tool semantics and execution within user space. The runtime workflow is systematically orchestrated: the llm-app (CLI/GUI) discovers tools via manifest-provided schemas and submits execution requests exclusively to the mcpd gateway. The gateway then queries the kernel via Generic Netlink to obtain an arbitration decision. Only upon receiving an ALLOW signal does mcpd dispatch the payload to per-app resident UDS services, subsequently reporting a TOOL_COMPLETE status back to the kernel for strict resource accounting. To guarantee system integrity, the design incorporates a fail-fast startup reconciliation mechanism utilizing canonical semantic hashing, which effectively prevents manifest-kernel semantic drift. Furthermore, the architecture is reinforced with kernel-side registries exposed via sysfs for observability, and persistent tool services safeguarded by length-prefixed JSON IPC and rigorous input validation.

尚需完成的任务 Work to do:

First, the arbitration model can be extended to support more granular governance, including hierarchical permission tiers, dynamic cost budgets, and adaptive backoff algorithms for deferred requests.
Second, deeper performance profiling such as syscall-level tracing is planned to rigorously quantify the IPC overhead and benchmark the architecture against traditional subprocess invocations under diverse concurrent workloads.
Finally, system observability will be further improved by expanding sysfs telemetry interfaces and developing standardized, automated test suites to facilitate reproducible evaluations in production environments.

存在问题 Problems:

There are two gaps remain before it can be considered complete and thesis-ready.

The first is security and trust-boundary clarity for high-risk tools (especially file operations). The current design correctly routes execution through a single gateway and uses manifests plus schema constraints, but the defense-grade security story still depends on clearly defined assumptions and systematic edge-case coverage. What is missing is a rigorous threat model (who is trusted, who can connect to which socket, what the attacker can do) and a set of adversarial/edge-case tests that demonstrate the boundary holds under realistic misuse scenarios.

The second is the report still needs stronger quantitative support on performance overhead and correctness under failure modes. A syscall-level comparative analysis would make the overhead of UDS framing and Netlink arbitration and dispatch concrete and comparable to baselines like subprocess invocation or direct syscall workflows.

拟采取的办法 Solutions:

First, to solidify the security and trust boundaries, a rigorous threat model and explicit trust assumptions will be formalized. At the implementation level, the system will enforce a strict, consistent sandboxing mechanism confined to the repository root. This containment strategy will be achieved by actively rejecting absolute paths and directory traversal attempts, as well as mitigating symlink escape vulnerabilities through strict realpath validation and the disabling of symlink resolutions for destructive operations. Additionally, semantic guardrails will be significantly tightened for high-risk actions such as file_delete. The robustness of these architectural boundaries will be empirically validated using targeted adversarial testing and edge-case injection scenarios to simulate realistic misuse.

Second, to strengthen the quantitative evaluation framework, the study will conduct a granular syscall-level overhead analysis utilizing system profiling utilities. This profiling will establish concrete performance metrics that will be directly compared against traditional subprocess-based and direct-invocation baselines. Furthermore, the correctness benchmarking suite will be significantly extended to cover diverse failure modes and exceptional states. This expansion includes validating the fail-fast termination logic during manifest-kernel hash mismatches, evaluating the system's backoff behavior during DEFER arbitration, and testing system resilience against IPC timeouts, partial execution failures, and service restart recovery. To ensure scientific rigor and reproducibility, all benchmarking procedures will be fully automated using standardized scripts designed for structured telemetry and metrics collection.

论文结构 Structure of the final report: (Chapter headings and section sub headings)

Chapter 1: Introduction
Chapter 2: Background
- **2.1 Traditional Operating System**
- **2.2 Existing AI Operating System**
- **2.3 Model Context Protocol**

Chapter 3: Design and Implementation
- **3.1 Overview**
- **3.2 Workflow and Protocols**
- **3.3 Kernel-Side Implementation**
- **3.4 User-Space Implementation**
- **3.5 Safety and Consistency Mechanisms**

Chapter 4: Results and Discussion
- **4.1 Experimental Setup**
- **4.2 Results**
- **4.3 Case Study**
- **4.4 Discussion**

Chapter 5: Conclusion and future work
- **5.1 Conclusion**
- **5.3 Future work**

Supervision log

北京邮电大学 本科毕业设计（论文）教师指导记录表

Project Supervision Log

学院 School	International School	**专业 Programme**	**Telecommunications Engineering with Management**		
姓 Family name	Li	**名 First Name**	Xiheng		
BUPT 学号 BUPT number	2022213192	**QM 学号 QM number**	221166471	**班级 Class**	2022215107
论文题目 Project Title	From Traditional Operating Systems to AI Agent Operating Systems: Exploring AI-Native OS through the Lens of the MCP Protocol				

Please record supervision log using the format below:

Date: dd-mm-yyyy
Supervision type: face-to-face meeting/online meeting/email/messages/other (please specify)
Summary:

Date: 07-11-2025
Supervision type: online meeting
Summary: discussed the project scope and agreed on the overall direction of the linux-mcp prototype, separating kernel governance from user space tool orchestration.

Date: 05-12-2025
Supervision type: online meeting
Summary: reviewed the system architecture, including the Generic Netlink control plane in kernel-mcp and the role of mcpd as the central execution gateway.

Date: 19-12-2025
Supervision type: online meeting
Summary: discussed the interaction flow between the kernel module, mcpd, and the userspace clients, and refined the responsibilities of each layer in the system.

Date: 16-01-2026
Supervision type: online meeting
Summary: discussed the design of manifest-based tool registration and the coordination between tool-app, mcpd, and the kernel registry.

Date: 24-01-2026
Supervision type: online meeting
Summary: discussed the service structure, including resident Unix domain socket services, handler dispatching, and safe file access constraints in the file management tools.

Date: 03-03-2026
Supervision type: online meeting
Summary: discussed the organization of tools into separate apps, including calculator, utility, settings, and file manager modules, to improve maintainability and semantic clarity.

Date: 10-03-2026
Supervision type: online meeting

Summary: discussed the latest progress on semantic hash binding, tool and app discovery in llm-app, and the overall end-to-end workflow from request submission to controlled tool execution.

Additional Appendices

A.1 Wire Protocol Reference

This section provides the complete field specifications and representative implementation fragments for the three inter-component protocols described in Section 3.4.

Complete llm-app to mcpd message reference. The `tool:exec` request carries the tool hash as a 64-character hexadecimal string. If a prior `DEFER` decision returned a `ticket_id`, the client includes it in a replay request:

Listing 1: tool:exec request and response messages (llm-app → mcpd).

```
// tool:exec request
{ "kind": "tool:exec",  "req_id": 42,
  "session_id": "sess-abc", "agent_id": "agent-1",
  "app_id": "sys_app",  "tool_id": 1,
  "tool_hash": "a3f2...c91b",        // 64-char SHA-256 hex
  "payload": {"path": "/tmp"},
  "approval_ticket_id": null }       // set on replay after DEFER

// tool:exec response (ALLOW)
{ "req_id": 42, "status": "ok",
  "result": {"os": "Linux 6.8", "cpu": "4-core"},
  "decision": "ALLOW", "reason": "admitted",
  "ticket_id": 0,  "t_ms": 3 }

// tool:exec response (DEFER)
{ "req_id": 42, "status": "error",
  "result": {}, "error": "deferred pending approval",
  "decision": "DEFER", "reason": "risk_tag:sensitive_read",
  "ticket_id": 7,  "t_ms": 1 }
```

The `approval_reply` message is issued by the client after the user confirms a deferred operation:

Listing 2: approval_reply message (llm-app → mcpd, issued after user confirms a deferred operation).

```
// approval_reply request
{ "sys": "approval_reply",  "session_id": "sess-abc",
  "ticket_id": 7,  "decision": "approve",
  "reason": "user confirmed",  "ttl_ms": 30000 }
```

mcpd to tool-app message schema. Once the kernel returns `ALLOW`, `mcpd` forwards the request to the relevant tool service over a per-application UDS connection using the minimal five-

field schema below. Tool services implement only this contract; they require no knowledge of sessions, kernel tickets, or the Netlink protocol. The `operation` field is the dispatch string a tool service routes on and is declared per-tool in the manifest; it is distinct from the `tool_id` (numeric, used by the kernel for registry indexing) and from the tool's human-readable `name`.

Field	Dir	Type	Description
`req_id`	req	*u64*	Per-invocation correlator
`agent_id`	req	*string*	Requesting agent identifier
`tool_id`	req	*u32*	Numeric tool identifier from the manifest
`operation`	req	*string*	Operation name within the tool
`payload`	req	*object*	Validated input matching the tool's schema
`req_id`	resp	*u64*	Must echo the request `req_id`
`status`	resp	*string*	`"ok"` or `"error"`
`result`	resp	*object*	Tool output on success; `{}` on failure
`error`	resp	*string*	Empty on success; error description on failure
`t_ms`	resp	*u32*	Tool-side execution time in milliseconds

Table 10: `mcpd–tool-app` JSON-RPC message fields.

Complete Netlink attribute encoding. All attributes are 4-byte aligned. String attributes are NUL-terminated and passed as raw bytes. The following Python fragment shows how `mcpd` encodes a `TOOL_REQUEST` message:

Listing 3: Python fragment: encoding a `TOOL_REQUEST` Generic Netlink message (mcpd → kernel).

```
import struct, socket, hashlib

FAMILY_NAME        = b"KERNEL_MCP\x00"
CMD_TOOL_REQUEST   = 10
ATTR_AGENT_ID,   ATTR_TOOL_ID     = 4,  2
ATTR_REQ_ID,     ATTR_TOOL_HASH   = 1,  19
ATTR_AGENT_BINDING, ATTR_AGENT_EPOCH = 28, 29

def build_nla(attr_type, data):
    # NLA header: 2-byte length, 2-byte type; 4-byte aligned
    hdr = struct.pack("HH", len(data) + 4, attr_type)
    pad = (4 - len(data) % 4) % 4
    return hdr + data + b"\x00" * pad

def encode_tool_request(agent_id, tool_id, req_id,
                        tool_hash, binding, epoch):
    attrs = (
```

```
        build_nla(ATTR_AGENT_ID,        agent_id.encode() + b"\x00") +
        build_nla(ATTR_TOOL_ID,         struct.pack("=I", tool_id))   +
        build_nla(ATTR_REQ_ID,          struct.pack("=Q", req_id))    +
        build_nla(ATTR_TOOL_HASH,       tool_hash.encode() + b"\x00") +
        build_nla(ATTR_AGENT_BINDING,   struct.pack("=Q", binding))   +
        build_nla(ATTR_AGENT_EPOCH,     struct.pack("=Q", epoch))
    )
    # Generic Netlink header: cmd (1 B), version (1 B), reserved (2 B)
    genl_hdr = struct.pack("BBH", CMD_TOOL_REQUEST, 1, 0)
    return genl_hdr + attrs
```

Table 11 lists the full `KERNEL_MCP` attribute set. The subset carried on the critical `TOOL_REQUEST`/`TOOL_DECISION` path is reproduced inline in Section 3.4.

Attribute	#	Type	Description
`REQ_ID`	1	*u64*	Per-invocation request identifier
`TOOL_ID`	2	*u32*	Numeric tool identifier
`AGENT_ID`	4	*string*	Agent identifier
`TOOL_HASH`	19	*string*	64-char hex SHA-256 of manifest fields
`BINARY_HASH`	33	*string*	SHA-256 of backend's `/proc/<pid>/exe`; composite with on-disk script bytes for interpreter-hosted tools; computed by `mcpd` at live probe time
`DECISION`	17	*u32*	Kernel verdict: 1=ALLOW, 2=DENY, 3=DEFER
`TICKET_ID`	22	*u64*	Approval ticket handle (DEFER path)
`APPROVAL_DECISION`	23	*u32*	1=approve, 2=deny, 3=revoke
`AGENT_BINDING`	28	*u64*	Session binding hash over `SO_PEERCRED`, computed by `mcpd`
`AGENT_EPOCH`	29	*u64*	Session opened-at epoch, gating cross-session replay

Table 11: Complete `KERNEL_MCP` TLV attribute set. All strings are NUL-terminated and 4-byte aligned. The per-tool catalog epoch (bumped when a tool's manifest is re-registered) is maintained internally by the kernel and is not transmitted as an attribute.

Semantic hash computation. The semantic hash used for tool identity is a full 256-bit SHA-256 digest computed over a canonical JSON serialisation of the following manifest fields: `tool_id`, `name`, `app_id`, `app_name`, `risk_tags`, `description`, `input_schema`, `examples`, `path_semantics`,

approval_policy, and operation. The endpoint and transport fields are deliberately excluded so that redeployment to a new socket path or transport does not invalidate the kernel registration; operation remains in-hash because it selects the RPC method actually executed on the backend.

Listing 4: Python fragment: computing the manifest-derived semantic hash used for tool identity.

```
import json, hashlib

SEMANTIC_FIELDS = [
    "tool_id", "name", "app_id", "app_name",
    "risk_tags", "description", "input_schema",
    "examples", "path_semantics", "approval_policy",
]

def semantic_hash(tool_manifest: dict) -> str:
    canonical = {f: tool_manifest[f] for f in SEMANTIC_FIELDS}
    blob = json.dumps(canonical,
                      sort_keys=True,
                      separators=(",", ":"),
                      ensure_ascii=True).encode("utf-8")
    return hashlib.sha256(blob).hexdigest()  # 64-char hex;
        never truncated
```

The full 64-character digest is used without truncation. A 32-bit prefix (8 hex characters) would be vulnerable to birthday collisions after approximately 2^{16} distinct tool registrations, which is a realistic size for a production deployment spanning multiple applications.

Complete manifest field specification.

A.2 System Comparison Table

A.3 Experiment Kernel Interface

The latency, stage-ablation, and extended-attack experiments in Chapter 4 rely on three kernel-side additions that live behind compile-time and module-parameter switches so that a single binary can serve both production and benchmark roles. This section documents their UAPI footprint and the peer-credential patch surfaced by the cross-UID attack rerun.

Noise-floor command: KMCP_CMD_NOOP. The paired ablation anchors every stage cost against a Generic Netlink round-trip floor measured by a minimum-work handler that returns immediately after command dispatch. The command is appended to the existing KERNEL_MCP family at a stable numeric ID so that older mcpd builds continue to work unchanged:

Field	In hash?	Description
Application-level fields		
`app_id`	✓	Unique application identifier string
`app_name`	✓	Human-readable application name
`transport`	✗	Must be `"uds_rpc"`
`endpoint`	✗	Absolute socket path under `/tmp/linux-mcp-apps/`
Tool-level fields		
`tool_id`	✓	Numeric identifier, unique within deployment
`name`	✓	Tool name presented to the LLM planner
`description`	✓	Natural-language description of tool behaviour
`risk_tags`	✓	Risk labels controlling kernel DEFER (e.g., `"sensitive_read"`)
`input_schema`	✓	JSON Schema object for payload validation
`examples`	✓	Example invocation payloads
`path_semantics`	✓	Path interpretation hints (e.g., `"repo_rel"`)
`approval_policy`	✓	Policy name triggering kernel DEFER; null for unrestricted tools
`operation`	✓	RPC method invoked on the backend; in-hash so a retarget changes tool identity
`timeout_ms`	✗	Per-invocation execution timeout (operational metadata)

Table 12: Manifest field specification. ✓ = included in the SHA-256 semantic identity hash; ✗ = excluded (operational metadata whose change should not invalidate a tool registration). Fields are grouped by level: application-level fields appear once per `app_id`; tool-level fields appear once per tool entry.

Listing 5: UAPI additions for the noise-floor command and the stage-ablation experiment flags (kernel-mcp/include/uapi/linux/kernel_mcp_schema.h).

```
/* New command: stable numeric ID, append-only. */
#define KERNEL_MCP_CMD_NOOP                15

/* Compile-time experiment flags carried in
   KERNEL_MCP_ATTR_EXPERIMENT_FLAGS
 * on KMCP_CMD_TOOL_REQUEST. Unknown bits are ignored. */
#define KERNEL_MCP_EXPERIMENT_SKIP_LOOKUPS  (1U << 0)
#define KERNEL_MCP_EXPERIMENT_SKIP_HASH     (1U << 1)
#define KERNEL_MCP_EXPERIMENT_SKIP_BINDING  (1U << 2)
#define KERNEL_MCP_EXPERIMENT_SKIP_TICKET   (1U << 3)
```

The `SKIP_*` flags are honoured only when the kernel is built with `CONFIG_KERNEL_MCP_EXPERIMENT=y`; production builds ignore the attribute entirely, so the mere presence of the UAPI constants does not weaken the enforcement path. Under the benign workload used by the ablation runner, the `SKIP_HASH` and `SKIP_BINDING` guards are statically dead code (short-circuited by a valid request before the flag is consulted) and their paired deltas correctly return zero to within noise;

System	Control-plane placement	Tool identity	State durability	Trust anchor & security boundary
`linux-mcp` (this work)	Kernel module + user-space gateway	SHA-256 of JSON manifest; kernel-verified at every invocation	Approval tickets and registry survive daemon crash (kernel-held)	**Kernel**; enforcement is independent of gateway correctness
`OpenClaw` [10]	User-space gateway (persistent process)	Skill name/version string in gateway registry	Lost on process restart	User-space process; bypassable via logic flaws [27, 28]
Harness Agent [22]	Cloud-hosted pipeline orchestrator	Pipeline step + RBAC connector identity	Cloud-hosted; platform-resilient	SaaS control plane; RBAC-based governance
AIOS / AgentOS [6, 7]	New OS-level agent kernel (proposed)	Agent/tool registration in LLM scheduler	Not evaluated (proposal stage)	New OS substrate; full redesign
LangChain / LangGraph [15, 16]	In-process user-space library	Tool name string; no cryptographic binding	Lost on process crash	Application process; no inter-invocation isolation
Apple Intelligence [23]	On-device OS + Private Cloud Compute	OS capability entitlements	High; OS-persistent services	Hardware attestation + OS entitlements; closed ecosystem

Table 13: Comparison of representative agent systems. The key differentiator for `linux-mcp` is the combination of cryptographic tool identity and kernel-held durable state, both absent from all user-space frameworks.

they are retained for methodology-consistency checks rather than as meaningful stage costs.

Peer-credential enforcement: `require_peer_cred`. The cross-UID attack rerun surfaced a defect in the stock kernel module: `kernel_mcp_cmd_tool_request` never compared the `NETLINK_CB(skb)` peer credential against the `uid` recorded at agent registration, which allowed a crafted-credential replay from a second UID to reach the tool-request handler untouched. The fix is exposed as a module parameter so that the A/B comparison in Chapter 4 could run against a single binary:

Listing 6: Peer-credential enforcement patch in `kernel_mcp_main.c`, gated behind the `require_peer_cred` module parameter.

```
static bool require_peer_cred = false;
module_param_named(require_peer_cred, require_peer_cred, bool,
    0644);
MODULE_PARM_DESC(require_peer_cred,
    "Reject TOOL_REQUEST if NETLINK_CB peer UID != agent->uid");

static int kernel_mcp_cmd_tool_request(struct sk_buff *skb,
                                       struct genl_info *info)
{
    if (require_peer_cred) {
```

```
        kuid_t peer = sock_i_uid(NETLINK_CB(skb).sk);
        if (!uid_eq(peer, agent->uid))
            return -EPERM;           /* cross-uid replay; reject */
    }
    /* ... existing admission pipeline ... */
}
```

The parameter defaults to `false` in the artefact published alongside this report so that the extended-attack experiment can exercise both the pre-patch and post-patch code paths without a rebuild; production deployments should set it to `true` via `/sys/module/kernel_mcp/parameters/require_` on module load. The check reads the peer credential on the same cache line that the existing `NETLINK_CB` access already touches, so the Chapter 4 latency and throughput numbers are unchanged whether the gate is on or off.

Artefact	Exposed as	Purpose
`KMCP_CMD_NOOP`	UAPI command (#15)	Generic Netlink round-trip noise-floor measurement. Returns immediately after dispatch; used by the kernel-ablation runner to anchor µs-level claims.
`SKIP_LOOKUPS`	`EXPERIMENT_FLAGS` bit 0	Bypasses the tool and agent registry walks. Used as the `skip_lookups` bar in Figure 8 and as the bypass mode in the paired registry+lookup delta in Table 7.
`SKIP_HASH`	`EXPERIMENT_FLAGS` bit 1	Bypasses the hash-mismatch guard. Guard body is dead under benign credentials; retained as a paired consistency check.
`SKIP_BINDING`	`EXPERIMENT_FLAGS` bit 2	Bypasses the binding-mismatch guard. Also dead under benign credentials; paired consistency check.
`SKIP_TICKET`	`EXPERIMENT_FLAGS` bit 3	Bypasses the approval-ticket issuance and consumption branch. Used as the `ticket_trigger_skip` mode to isolate the approval-ticket body cost.
`require_peer_cred`	Module parameter (`bool`, 0644)	Enables the `NETLINK_CB` peer-UID check in `kernel_mcp_cmd_tool_request`. Default `false` so that the cross-UID experiment can toggle it at runtime; should be set to `true` in production.

Table 14: Experiment-only kernel switches added for the Chapter 4 rerun. All four `SKIP_*` bits are ignored when the kernel is built without `CONFIG_KERNEL_MCP_EXPERIMENT=y`; `require_peer_cred` is runtime-toggleable on any build.

Summary of experiment-only switches.

Risk and Environmental Impact Assessment

Risk and Environmental Impact Assessment

This project investigates `linux-mcp`, a kernel-assisted control-plane architecture for AI tool invocation. Its goal is to improve the safety and observability of tool use in AI-agent systems by separating semantic interpretation and tool execution in user space from admission control and approval handling in the kernel. Because the system spans an LLM-facing application, a user-space gateway, kernel mechanisms, and external tools, it introduces technical, security, and resource-related risks. A formal risk and environmental impact assessment is therefore necessary.

Following the prescribed method, each risk is assigned a likelihood level L and a consequence level C, and the overall score is calculated as $R = L \cdot C$. This assessment considers risks to project completion, risks of harm to systems or organisations, environmental impact, and losses of time or computational resources.

Project and technical risks

A primary technical risk is kernel-level instability. Since `linux-mcp` moves part of the control plane into the Linux kernel, faults in Netlink communication, synchronization, or state management may lead to crashes or inconsistent behaviour. This risk is assigned moderate likelihood ($L = 3$) and major consequence ($C = 4$), giving $R = 12$. The mitigation is to keep the kernel scope narrow and rely on strict interfaces, staged testing, and stress testing.

A second risk is the possibility that the project does not demonstrate a convincing advantage over strong user-space baselines. The value of `linux-mcp` depends not only on correctness but also on showing that kernel placement provides stronger control properties than user-space mediation alone. This risk is assigned likely likelihood ($L = 4$) and very serious consequence ($C = 3$), giving $R = 12$. It should be mitigated by evaluation against both conventional and hardened user-space baselines under representative performance and adversarial workloads.

A third risk is cross-component integration failure. The system includes an LLM-side application, the `mcpd` daemon, the kernel control plane, and tool-side services, and mismatches between these components may create failures that are difficult to isolate. This risk is assigned moderate likelihood ($L = 3$) and serious consequence ($C = 2$), giving $R = 6$. This can be reduced through explicit protocol definitions, schema synchronization, and end-to-end validation.

Security and operational risks

A central operational risk is unsafe or unauthorized tool invocation. Since `linux-mcp` mediates LLM-triggered access to tools with possible side effects, an error in the admission path could allow an action that should have been denied or deferred. This risk is assigned moderate likelihood ($L = 3$) and major consequence ($C = 4$), giving $R = 12$. Mitigation includes deny-by-default

admission, explicit approval binding, auditable state transitions, and attack-oriented testing.

A second risk is overestimating the system's security guarantees. Kernel placement can strengthen enforcement of specific invariants, but it does not eliminate threats arising from semantic errors in user space, unsafe tool implementations, or incomplete policies. This risk is assigned moderate likelihood ($L = 3$) and very serious consequence ($C = 3$), giving $R = 9$. The mitigation is to define the threat model precisely and to state clearly which protections are provided by kernel enforcement and which remain outside its scope.

A further risk is degraded responsiveness under load. Additional arbitration and approval steps may increase latency or reduce throughput, especially when multiple agents or tools are active concurrently. This risk is assigned moderate likelihood ($L = 3$) and serious consequence ($C = 2$), giving $R = 6$. It should be mitigated through latency analysis, concurrency benchmarking, and optimization of common low-risk paths.

Environmental and resource impact

The direct environmental impact of `linux-mcp` is limited because the project is software-based and does not involve substantial material waste. However, development and evaluation still consume computational resources. Repeated benchmarking, adversarial testing, virtual-machine execution, and debugging of kernel-space behaviour all increase CPU time and electricity consumption. This environmental risk is assigned likely likelihood ($L = 4$) and serious consequence ($C = 2$), giving $R = 8$. The appropriate mitigation is careful experiment planning, automation of result collection, and avoidance of redundant reruns.

A practical project-management risk is loss of development time and compute resources due to unstable experiments, failed integration, or repeated debugging. This risk is assigned likely likelihood ($L = 4$) and serious consequence ($C = 2$), giving $R = 8$. Mitigation includes reproducible scripts, checkpointing of experiment outputs, staged deployment, and careful configuration management.

Another resource-related risk is dependence on external tooling or infrastructure. The project may rely on LLM APIs, network configuration, proxies, virtual machines, or other software dependencies during development and evaluation. This risk is assigned moderate likelihood ($L = 3$) and serious consequence ($C = 2$), giving $R = 6$. This risk can be reduced by separating the core evaluation from optional cloud dependencies and by supporting local or mocked execution paths where possible.

Table 15: Summary of identified risks and mitigation strategies for `linux-mcp`.

Risk	*L*	*C*	*R*	**Rating**	**Mitigation**
Kernel-level instability	3	4	12	Significant	Narrow kernel scope, strict interfaces, staged testing, stress testing
Weak justification over user-space baselines	4	3	12	Significant	Strong baselines, adversarial evaluation, quantitative comparison
Cross-component integration failure	3	2	6	Moderate	Explicit protocols, schema synchronization, end-to-end validation
Unsafe or unauthorized tool invocation	3	4	12	Significant	Deny-by-default admission, approval binding, auditability, misuse testing
Overestimated security guarantees	3	3	9	Significant	Precise threat model, careful claims, clear scope of guarantees
Added latency or reduced throughput	3	2	6	Moderate	Latency analysis, concurrency evaluation, fast-path optimization
Additional compute and energy use	4	2	8	Significant	Efficient experiment planning, automation, fewer redundant reruns
Loss of development time and compute resources	4	2	8	Significant	Reproducible workflows, checkpointing, staged deployment
External dependency or infrastructure failure	3	2	6	Moderate	Local or mocked backends, dependency isolation, reduced cloud reliance